



Escuela
Politécnica
Superior

Aplicación colaborativa para propiciar el cuidado del medio ambiente

Grado en Ingeniería Informática



Trabajo Fin de Grado

Autor:

Erardo Aldana Pessoa

Tutor/es:

Dr. José Luis Sánchez Romero

Junio 2026



Universitat d'Alacant
Universidad de Alicante

Agradecimientos

A mi tutor, José Luis Sánchez Romero, por la disponibilidad, los consejos técnicos y la libertad que me ha dado para definir el alcance y el *stack* del proyecto. Sin esa libertad, EcoAlerta no habría salido como ha salido.

A mi familia, por el apoyo incondicional durante todos los años del Grado.

A los compañeros con los que he compartido asignaturas, prácticas y discusiones; aprender al lado de gente competente es la mejor manera de aprender.

A la Escuela Politécnica Superior de la Universidad de Alicante por la formación recibida.

A mi novia Ana, por ayudarme a estructurar el trabajo de manera más óptima y por ser un apoyo constante durante el desarrollo del proyecto.

Resumen

EcoAlerta es una aplicación web progresiva (PWA) concebida como red social cívica especializada en medio ambiente. Permite a cualquier ciudadano reportar incidencias medioambientales (vertidos ilegales, contaminación, incendios forestales, animales abandonados o maltratados, daños a espacios naturales, entre otras) con foto y GPS, las clasifica por categoría y severidad, y las delega a la entidad responsable correcta (ayuntamiento, SEPRONA, bomberos, policía local) mediante un flujo de estados trazable. Los ciudadanos consultan el mapa público sin registrarse y, al autenticarse, votan, comentan y siguen los reportes de otros usuarios, creando un componente social que ayuda a priorizar las incidencias más relevantes.

He desarrollado el proyecto en seis *sprints* de una semana con enfoque iterativo e incremental, aplicando *Spec-Driven Development* (SpecKit) para combinar especificaciones formales y asistencia de agentes de IA en el IDE Google Antigravity. El uso de IA está declarado de forma explícita en la memoria; las decisiones de diseño, la revisión de código y la responsabilidad final son mías.

Tecnológicamente el sistema tiene tres capas en Docker Compose: SPA en React 18 con Tailwind y Leaflet sobre OpenStreetMap (instalable como PWA, con soporte *offline*), API REST en Node.js 20 + Express 4 (con JWT dual token, 2FA TOTP, *rate limiting* y Cloudflare Turnstile), y PostgreSQL 16 con PostGIS 3.4 (índice GIST y ST_DWithin para consultas geoespaciales). El modelo tiene 13 entidades que cubren incidencias, fotos, comentarios, votos, seguimientos, historial de estados, notificaciones y un *audit log* de seguridad.

He definido 46 requisitos funcionales en cinco módulos (autenticación, incidencias, mapa, administración, social) y 12 requisitos no funcionales. La API expone 35 *endpoints* REST documentados con Swagger. La validación combina pruebas unitarias y de integración con Jest+Supertest, y pruebas E2E con Playwright en tres navegadores; todo en un *pipeline* de GitHub Actions.

Los resultados confirman la viabilidad del sistema: despliegue reproducible con un comando, cobertura del *backend* por encima del 60 %, búsquedas geoespaciales en un radio de 5 km por debajo de 500 ms y validación de todos los requisitos funcionales de prioridad alta.

Palabras clave: aplicación cívica, medio ambiente, PostGIS, PWA, React, Node.js, JWT, Docker, Spec-Driven Development, OpenStreetMap.

Abstract

EcoAlerta is a Progressive Web Application (PWA) conceived as a civic social network specialised in environmental issues. It allows any citizen to report environmental incidents (illegal dumping, pollution, wildfires, abandoned or mistreated animals, damage to natural areas, among other categories) with a photo and GPS coordinates. The system classifies each report by category and severity, and delegates it to the appropriate responsible entity (city council, SEPRONA, fire department, local police) through a traceable state flow. Citizens consult the public map without registering and, once authenticated, vote, comment and follow other users' reports, creating a social component that helps prioritise the most relevant incidents.

The project was developed in six one-week sprints following an iterative and incremental approach, applying Spec-Driven Development (SpecKit) to combine formal specifications with AI agent assistance in the Google Antigravity IDE. The use of AI is explicitly declared in the report; design decisions, code review and final responsibility belong to the author.

Technologically, the system has three layers deployed with Docker Compose: a SPA in React 18 with Tailwind and Leaflet over OpenStreetMap (installable as a PWA with offline support), a REST API in Node.js 20 + Express 4 (with JWT dual token, TOTP 2FA, rate limiting and Cloudflare Turnstile), and PostgreSQL 16 with PostGIS 3.4 (GIST index and `ST_DWithin` for geospatial queries). The data model has 13 entities covering incidents, photos, comments, votes, follows, status history, notifications and a security audit log.

46 functional requirements were defined across five modules (authentication, incidents, map, administration, social) plus 12 non-functional requirements. The REST API exposes 35 endpoints documented with Swagger. Validation combines unit and integration tests with Jest+Supertest, and E2E tests with Playwright across three browsers; the entire pipeline runs on GitHub Actions.

Results confirm the system's viability: reproducible deployment with one command, backend test coverage above 60%, geospatial searches within a 5 km radius under 500 ms, and validation of all high-priority functional requirements.

Keywords: civic application, environment, PostGIS, PWA, React, Node.js, JWT, Docker, Spec-Driven Development, OpenStreetMap.

Índice general

Agradecimientos	iii
Resumen	v
Abstract	vii
Índice de figuras	xiii
Índice de tablas	xv
Índice de listados	xvii
1 Introducción	1
1.1 Motivación y contexto	1
1.2 Objetivos	2
1.3 Uso de herramientas de inteligencia artificial	2
1.4 Estructura de la memoria	3
2 Estado del arte	5
2.1 Contexto: aplicaciones cívicas geolocalizadas	5
2.2 Aplicaciones analizadas	5
2.2.1 FixMyStreet	5
2.2.2 Línea Verde	6
2.2.3 iNaturalist	7
2.2.4 SeeClickFix	7
2.2.5 Epicollect5	8
2.3 Comparativa funcional	8
2.4 Tecnologías evaluadas	9
2.5 Posicionamiento de EcoAlerta	10
3 Análisis y diseño	13
3.1 Metodología de desarrollo	13
3.2 Actores del sistema	14
3.3 Requisitos	14
3.3.1 Requisitos funcionales	14
3.3.2 Requisitos no funcionales	16

3.4	Casos de uso	17
3.5	Arquitectura del sistema	17
3.5.1	Stack tecnológico	20
3.6	Modelo de datos	20
3.6.1	Entidades principales	21
3.6.2	Rendimiento geoespacial	22
3.7	Flujo de estados de una incidencia	22
3.8	Diseño de la API REST	23
3.9	Diseño de la interfaz de usuario	25
4	Implementación	27
4.1	Entorno y herramientas	27
4.2	Estructura del repositorio	27
4.3	Desarrollo por sprints	28
4.3.1	Sprint 1: autenticación y BD	28
4.3.2	Sprint 2: CRUD de incidencias	29
4.3.3	Sprint 3: mapa interactivo	30
4.3.4	Sprint 4: admin y delegación	31
4.3.5	Sprint 5: testing y PWA	31
4.3.6	Sprint 6: <i>security hardening</i>	31
4.4	<i>Frontend</i> : notas adicionales	32
4.5	Despliegue con Docker Compose	32
4.6	Integración continua	32
4.7	Capturas de pantalla	33
4.8	Retos técnicos resueltos	34
4.9	Asistencia de IA en la implementación	35
5	Evaluación	37
5.1	Pruebas automatizadas del <i>backend</i>	37
5.2	Pruebas <i>end-to-end</i> con Playwright	38
5.3	Validación del cumplimiento de requisitos	38
5.4	Pruebas de usabilidad: protocolo y trabajo futuro	40
5.5	Valoración global	41
6	Conclusiones y trabajo futuro	43
6.1	Logros	43
6.2	Limitaciones	43
6.3	Trabajo futuro	44

6.4 Reflexión	45
Guía de instalación	49
Manual de usuario	53
1 Ciudadano anónimo	53
2 Ciudadano registrado	54
3 Administrador	56
4 Entidad responsable	57
5 Accesibilidad	57
Modelo de negocio preliminar	59

Índice de figuras

3.1	Diagrama de casos de uso. Elaboración propia a partir de RF-01–RF-46. .	18
3.2	Arquitectura de componentes. Elaboración propia.	19
3.3	Modelo entidad-relación (13 tablas). Elaboración propia.	21
3.4	Diagrama de estados de la incidencia. Cada transición se guarda en <code>incident_status_history</code> . Elaboración propia.	23
3.5	Pantalla principal de EcoAlerta: mapa centrado en Alicante con marcadores coloreados por severidad y panel lateral de filtros. Captura propia.	25
4.1	Pantalla principal con el mapa de incidencias y el panel lateral de filtros. Captura propia.	33
4.2	Formulario de reporte de una incidencia, con categoría, severidad y ubicación. Captura propia.	33
4.3	Detalle de una incidencia con sus fotos, votos, comentarios y estado. Captura propia.	34
4.4	Dashboard del administrador con gráficos agregados (Recharts). Captura propia.	34
1	Vista del mapa en dispositivo móvil (<i>viewport</i> 375 px). Captura propia. .	54
2	Formulario de registro del ciudadano. Captura propia.	55
3	Modal de activación del segundo factor (2FA) con código QR. Captura propia.	55
4	Panel de gestión de incidencias del administrador. Captura propia. . . .	56

Índice de tablas

2.1	Comparativa funcional. ✓ sí, ~ parcial, × no.	8
2.2	Decisiones tecnológicas por capa.	10
3.1	Actores del sistema.	14
3.2	Requisitos funcionales (RF-01 a RF-46).	14
3.3	Requisitos no funcionales.	16
3.4	Stack tecnológico.	20
3.5	Endpoints principales.	24
4.1	Resumen de los seis sprints.	28
5.1	Cobertura final del <i>backend</i>	38
5.2	Pruebas E2E con Playwright.	38
5.3	Resumen del cumplimiento de requisitos funcionales.	39
5.4	Cumplimiento de requisitos no funcionales.	40
1	Planes propuestos.	60

Índice de Códigos

3.1	SQL	22
3.2	Índice GIST y consulta geoespacial por radio sobre <code>incidents</code> . Elaboración propia.	22
4.1	JavaScript	29
4.2	Generación de <i>access</i> y <i>refresh tokens</i> JWT. Elaboración propia.	29
4.3	JavaScript	29
4.4	<i>Middleware</i> de control de acceso por rol. Elaboración propia.	29
4.5	JavaScript	30
4.6	Búsqueda de incidencias cercanas con filtros dinámicos y orden por distancia. Elaboración propia.	30

1 Introducción

1.1 Motivación y contexto

España vive una década compleja en lo medioambiental: incendios forestales recurrentes ligados al cambio climático, contaminación atmosférica en ciudades, vertidos en suelos rústicos, maltrato animal. La Ley 7/2022 de residuos, la Ley de bienestar animal y el Pacto Verde Europeo definen un marco exigente, pero su efectividad depende de algo concreto: **detectar a tiempo lo que pasa sobre el terreno**.

Los datos son claros. El Ministerio del Interior registra más de 10.000 incendios forestales al año en España. El SEPRONA tramita más de 130.000 actuaciones anuales por infracciones medioambientales. Una buena parte de estas incidencias llega tarde a los canales oficiales, o no llega.

A la vez, la ciudadanía tiene en el bolsillo lo que necesitaría para reportarlas: el INE estimó en 2023 que el 97,7 % de los hogares españoles tiene Internet y que más del 95 % dispone de un *smartphone* con cámara y GPS. El problema es que los canales tradicionales (el teléfono 112 de emergencia, las oficinas presenciales y los formularios web genéricos sin geolocalización) no aprovechan ese flujo y no devuelven seguimiento al denunciante.

En cambio, existen aplicaciones internacionales que sí cubren este hueco. *FixMyStreet* en el Reino Unido lleva desde 2007 acumulando más de dos millones de incidencias reportadas por ciudadanos. *iNaturalist* agrega 180 millones de observaciones de biodiversidad. *Línea Verde* ofrece en España un canal municipal de quejas en más de 300 municipios. Ejemplos que se analizarán más detalladamente en el Capítulo 2. Pero ninguna combina, a la vez, reporte geolocalizado con foto, delegación automatizada a la entidad responsable correcta (que no siempre es el ayuntamiento), niveles de severidad ligados al impacto ambiental, y un componente social que ayude a priorizar.

De este contexto nace el presente trabajo. **EcoAlerta** es una aplicación web progresiva (PWA) que combina red social cívica y reporte medioambiental: cualquier ciudadano reporta una incidencia con foto y GPS en menos de un minuto, el sistema la clasifica por categoría y severidad, la delega a la entidad responsable (Ayuntamiento, SEPRONA, Bomberos, Policía Local) mediante un flujo de estados trazable, y otros usuarios pueden votar y seguir su evolución. Cualquier persona consulta el mapa público sin registrarse; pero las acciones con responsabilidad (crear, votar, comentar) requieren cuenta y quedan trazadas.

1.2 Objetivos

Teniendo en cuenta el contexto de este proyecto, el objetivo general que se pretende lograr es:

Diseñar, implementar y validar una aplicación web progresiva que facilite a la ciudadanía el reporte geolocalizado de incidencias medioambientales y permita su delegación automatizada a entidades responsables, con flujo de estados trazable y componente social.

A su vez, se divide en ocho objetivos específicos:

1. Analizar el estado del arte de aplicaciones cívicas medioambientales e identificar huecos de cobertura.
2. Definir una arquitectura escalable basada en contenedores, separando presentación, lógica y datos, con soporte geoespacial.
3. Diseñar un modelo de datos relacional con extensión PostGIS que represente incidencias, fotos, actores, flujos y eventos sociales con *audit trail*.
4. Implementar una API REST documentada con OpenAPI/Swagger, con control de acceso por roles y *rate limiting*.
5. Desarrollar una interfaz web progresiva *mobile-first*, instalable, con soporte *offline* básico, mapa interactivo y conformidad con WCAG 2.1 AA.
6. Incorporar mecanismos de seguridad propios de aplicaciones reales: JWT con rotación de tokens, 2FA, *captcha* invisible, *audit log* y cifrado de secretos.
7. Validar el sistema con una pirámide de pruebas (unitarias, integración, E2E) y pruebas de usabilidad con usuarios reales.
8. Producir documentación reproducible (despliegue con un comando, API documentada, manual de usuario) y un estudio preliminar de modelo de negocio.

1.3 Uso de herramientas de inteligencia artificial

Para este proyecto se han usado herramientas de IA generativa, declaradas explícitamente en este apartado, siguiendo lo que indica la Universidad de Alicante sobre integridad académica cuando la asistencia está permitida y se reconoce. Esta sección recoge qué he usado, para qué, y dónde se han tomado las decisiones propias.

Herramientas

Se han usado los modelos *Claude* (Anthropic) y *Gemini* (Google), este último integrado en el IDE *Google Antigravity* para programación asistida por agentes.

Tareas asistidas

La IA ha ayudado con la generación inicial de *boilerplate* (estructura de rutas Express, componentes React, configuraciones Docker), refactorización de fragmentos ya escritos, generación de pruebas Jest a partir del contrato de los servicios, redacción de borradores de la memoria a partir de la especificación, y revisión ortográfica y de estilo.

Tareas propias

La idea, el alcance y los objetivos del proyecto. La elección del *stack* y del modelo de datos. El análisis de las cinco aplicaciones de referencia: descarga, uso real y observaciones propias. Las pruebas de usabilidad. La revisión de cada bloque de código generado y su validación con ejecución local. La argumentación de los capítulos de análisis, evaluación y conclusiones.

Auditoría

El proyecto vive en un repositorio Git público con *commits* en convención *Conventional Commits*, lo que permite reconstruir el proceso. Las especificaciones de cada *sprint* están en *docs/sprints/* y se redactaron antes de pedir asistencia al agente. Asumo la responsabilidad de explicar y justificar en la defensa cualquier decisión de diseño, fragmento de código o párrafo de la memoria, sin distinción de cómo se generó inicialmente.

Mi posición

El valor del TFG destaca por tres cosas que la IA no aporta: la identificación del hueco de cobertura que tapa EcoAlerta, el diseño integrado del flujo ciudadano-administrador-entidad, y la ingeniería de requisitos que ha convertido una idea en 46 RF verificables. La IA es una herramienta de productividad; las decisiones y la responsabilidad son mías.

1.4 Estructura de la memoria

El presente trabajo se organiza de la siguiente manera. El Capítulo 2 analiza cinco aplicaciones de referencia y compara las tecnologías evaluadas. El Capítulo 3 presenta requisitos, casos de uso, arquitectura, modelo de datos y diseño de la API. El Capítulo 4 describe el desarrollo en seis *sprints*, con extractos de código y decisiones técnicas. El Capítulo 5 recoge resultados de pruebas y validación de requisitos. El Capítulo 6 cierra con logros, limitaciones

y trabajo futuro. Los anexos incluyen guía de instalación, manual de usuario y un análisis preliminar de modelo de negocio.

2 Estado del arte

Este capítulo revisa aplicaciones cívicas relevantes para EcoAlerta, compara las tecnologías candidatas para cada capa y justifica las decisiones tomadas en el diseño del Capítulo 3.

2.1 Contexto: aplicaciones cívicas geolocalizadas

Las aplicaciones cívicas con reporte geolocalizado son parte del movimiento *civic tech* y de lo que la literatura llama *ciudadanos como sensores* (Goodchild, 2007; Haklay, 2013). Tres innovaciones las hicieron viables a partir de mediados de los 2000:

1. *Smartphones* con cámara y GPS (iPhone 2007, Android 2008): cualquier ciudadano puede generar evidencia fotográfica geolocalizada en tiempo real.
2. OpenStreetMap (2004): cartografía libre, sin dependencia de proveedores comerciales.
3. Datos abiertos y leyes de transparencia (en España, Ley 19/2013): marco para el intercambio bidireccional ciudadano–administración.

A partir de ahí se consolidan dos familias: las **generalistas** (FixMyStreet, SeeClickFix, que cubren cualquier incidencia urbana) y las **especializadas** (iNaturalist para biodiversidad, Línea Verde para reporte municipal en España). EcoAlerta se sitúa en un hueco intermedio: especializada en medio ambiente, con componente social.

2.2 Aplicaciones analizadas

Se han seleccionado cinco aplicaciones por relevancia, heterogeneidad y cobertura del espectro de la *civic tech* aplicada al medio ambiente: FixMyStreet, Línea Verde, iNaturalist, SeeClickFix y Epicollect5. El análisis de cada una se ha realizado a partir de la documentación oficial, las páginas web del producto y reseñas públicas de usuarios, complementadas con la observación de capturas de pantalla y vídeos demostrativos disponibles en línea. A continuación se describe cada una de ellas.

2.2.1 FixMyStreet

Desarrollada en 2007 por *mySociety* (mySociety, 2024), organización británica sin ánimo de lucro especializada en *civic tech*. El código es abierto bajo licencia AGPL-3.0 y la plataforma

ha sido desplegada en cientos de ayuntamientos del Reino Unido, así como en versiones locales para Bruselas, Noruega o Suiza.

Funcionalidades principales. Reporte ciudadano de incidencias urbanas (baches, alumbrado, mobiliario dañado, vertidos) con foto opcional y ubicación geoespacial, listado público de reportes por zona, panel administrativo para los ayuntamientos contratantes, notificaciones de cambio de estado al autor del reporte y duplicado detectado automáticamente por proximidad.

Flujo de uso. El usuario abre la web móvil, introduce una dirección o selecciona un punto en el mapa, escoge una categoría y describe la incidencia. La plataforma encamina el reporte al ayuntamiento competente según la ubicación del marcador y le asigna un identificador público de seguimiento.

Tecnología. Aplicación web con diseño *mobile-friendly*. *Backend* en Perl con Plack y PostgreSQL. Cartografía sobre OpenStreetMap.

Aportaciones de referencia para EcoAlerta. Modelo de delegación municipal por ubicación geoespacial, flujo de estados visible para el ciudadano, identificador público de seguimiento y comunidad de usuarios.

Limitaciones respecto al objetivo de EcoAlerta. Enfoque generalista (urbano) sin especialización medioambiental. No contempla entidades responsables distintas del ayuntamiento (SEPRONA, bomberos, ONG). No incluye severidad con impacto en la priorización ni componente social significativo (votos, seguidores).

2.2.2 Línea Verde

Servicio comercial de la empresa española *Línea Verde Configuración y Servicios Medioambientales S.A.* (Línea Verde Smart City, 2024), contratado por más de 300 municipios españoles. Funciona bajo modelo B2G *white-label*: cada Ayuntamiento adquiere el servicio y obtiene su propia versión personalizada con marca municipal.

Funcionalidades principales. Reporte de quejas e incidencias municipales con foto y ubicación, consulta del calendario de recogida de residuos, búsqueda de puntos de reciclaje, alertas sobre cortes de servicios y avisos del ayuntamiento. La rama medioambiental cubre vertidos, animales muertos en vía pública, mobiliario urbano dañado y problemas de jardinería.

Flujo de uso. Aplicación móvil nativa (iOS/Android) y portal web. El ciudadano selecciona el tipo de incidencia, describe el problema, adjunta foto y envía. El reporte queda en cola del ayuntamiento contratante, que lo gestiona desde su propio panel y notifica al usuario cuando cambia el estado.

Tecnología. Aplicación móvil nativa publicada en App Store y Google Play, con portal web complementario. *Backend* y arquitectura no documentados públicamente al tratarse de

software propietario.

Aportaciones de referencia para EcoAlerta. Modelo *white-label* viable para administraciones locales españolas. Estructura de categorización compatible con la legislación municipal española. Notificaciones de cambio de estado al ciudadano.

Limitaciones respecto al objetivo de EcoAlerta. Software propietario sin código fuente público; cada despliegue queda aislado en el ámbito municipal contratante; el reporte no se delega a entidades especializadas distintas del propio ayuntamiento; ausencia de componente social entre ciudadanos.

2.2.3 iNaturalist

Plataforma global de ciencia ciudadana para observación de biodiversidad nacida en 2008 en la Universidad de California (California Academy of Sciences and National Geographic Society, 2024). Actualmente está gestionada conjuntamente por la *California Academy of Sciences* y la *National Geographic Society*. Acumula más de 180 millones de observaciones aportadas por una comunidad internacional de naturalistas y voluntarios.

Funcionalidades principales. Subida de observaciones de fauna y flora con foto y geolocalización, identificación automática de especies mediante una red neuronal entrenada con el corpus comunitario, validación comunitaria de las identificaciones, proyectos colaborativos por región o ecosistema y exportación de datos para investigación científica.

Flujo de uso. El usuario abre la aplicación, fotografía un ejemplar, la app sugiere especies candidatas, el usuario confirma o ajusta y publica. Otros usuarios pueden validar la identificación, lo que eleva el grado de confianza de la observación hasta «calidad de investigación».

Tecnología. Aplicación móvil nativa y web. *Backend* en Ruby on Rails. Base de datos PostgreSQL con extensión PostGIS para datos geoespaciales. Elasticsearch para búsquedas. Servicio de visión por computador propio para reconocimiento de imagen.

Aportaciones de referencia para EcoAlerta. Comunidad activa que valida los reportes, perfil público del usuario con histórico de observaciones, sistema de proyectos por región o temática, uso de PostGIS para consultas geoespaciales (decisión arquitectónica equivalente a la adoptada en EcoAlerta).

Limitaciones respecto al objetivo de EcoAlerta. Enfoque en biodiversidad, no en incidencias medioambientales. No contempla delegación a entidades responsables ni flujo de estados administrativo. No prioriza por severidad ni por urgencia de actuación.

2.2.4 SeeClickFix

Fundado en 2008 en New Haven (EE.UU.), referente del reporte cívico americano. Adquirido por CivicPlus en 2019, sigue operando en municipios norteamericanos.

Funcionalidades. Reporte geolocalizado con foto, categorías estandarizadas, flujo de estados (recibido, reconocido, en curso, resuelto), panel administrativo, integración con sistemas *backend* municipales vía API.

Tecnología. App móvil nativa (iOS/Android) y web. API documentada para integraciones.

Relevancia para EcoAlerta. Valida el modelo B2G con flujo de estados trazable. Su limitación es el foco urbano exclusivo: no aborda incendios forestales, fauna silvestre ni daños a espacios naturales.

2.2.5 Epicollect5

Proyecto del Imperial College London para recolección de datos de campo en investigación científica. Usado en epidemiología, conservación y ecología.

Funcionalidades. Formularios personalizados, recolección *offline* con sincronización diferida, exportación en CSV/JSON. Gratuito para investigación.

Tecnología. App móvil y web. *Stack* no documentado públicamente.

Relevancia para EcoAlerta. Su robustez *offline-first* es referencia para el RF-19. Su enfoque genérico lo hace demasiado amplio para el ciudadano medio, pero útil para ONG conservacionistas que pueden configurar formularios específicos.

2.3 Comparativa funcional

Tabla 2.1: Comparativa funcional. ✓ sí, ~ parcial, × no.

Funcionalidad	FixMy-Street	Línea Verde	iNaturalist	SeeClick-Fix	Epi-collect	Eco-Alerta
Reporte geolocalizado con foto	✓	✓	✓	✓	✓	✓
Enfoque medioambiental específico	×	~	✓	×	~	✓
Delegación a entidades	~	✓	×	✓	×	✓
Flujo de estados trazable	✓	~	×	✓	×	✓
Componente social (votos, comentarios)	~	×	✓	~	×	✓
Perfil público de usuario	×	×	✓	~	×	✓
Acceso anónimo de consulta	✓	✓	✓	✓	~	✓

Funcionalidad	FixMy-Street	Línea Verde	iNaturalist	SeeClick-Fix	Epi-collect	Eco-Alerta
Niveles de severidad explícitos	~	×	×	~	✓	✓
<i>Audit trail</i> completo	×	×	×	~	×	✓
PWA instalable	~	×	×	×	~	✓
<i>Offline</i>	×	×	✓	×	✓	✓
2FA / <i>audit</i> de seguridad	×	×	×	×	×	✓
<i>Open source</i>	✓	×	✓	×	~	✓

Ninguna aplicación cubre simultáneamente las seis dimensiones que busca EcoAlerta: enfoque medioambiental, delegación a entidades heterogéneas (no solo municipales), flujo trazable, social significativo, severidad con impacto en priorización y *audit trail*. Ese hueco es el punto de partida del proyecto (objetivo 1).

2.4 Tecnologías evaluadas

Para cada capa consideré varias opciones. La Tabla 2.2 resume las decisiones y por qué.

Tabla 2.2: Decisiones tecnológicas por capa.

Capa	Elección vs alternativas	Razón
Plataforma	PWA vs app nativa	PWA con una sola <i>code base</i> , distribución por URL, instalable. La nativa quedaría como trabajo futuro.
Frontend	React vs Vue, Angular	React tiene la mejor integración con Leaflet (react-leaflet), gran mercado laboral en España, y ya conocía la herramienta.
BD	PostgreSQL+PostGIS vs MongoDB	PostGIS aporta ST_DWithin , índices GIST y transacciones. El modelo relacional encaja mejor con las relaciones del dominio. MongoDB obligaría a denormalizar.
Mapa	Leaflet+OSM vs Google Maps	Leaflet es <i>open source</i> , sin coste ni dependencia comercial. Coherente con la filosofía <i>civic tech</i> de las apps de referencia.
Metodología	SDD vs Scrum clásico	Para un solo desarrollador con asistencia IA, la especificación previa funciona como guía al agente y como base para revisar. Más útil que un Scrum tradicional pensado para equipos.

2.5 Posicionamiento de EcoAlerta

Una vez analizadas todas ellas, hay que tener en cuenta que EcoAlerta combina elementos de varias fuentes:

- Flujo de estados trazable de FixMyStreet y SeeClickFix, ampliado con validación, marcado de duplicados y escalado por *timeout*.
- Delegación B2G multi-entidad de Línea Verde, generalizada a entidades no municipales (SEPRONA, bomberos, ONG).
- Componente social de iNaturalist (seguimiento, votos, comentarios), adaptado al ámbito medioambiental con la distinción de comentarios oficiales.

- *Offline* robusto al estilo Epicollect5, con Service Worker e IndexedDB.
- Mecanismos de seguridad propios (2FA TOTP, *audit log*, Cloudflare Turnstile) ausentes en las cinco apps analizadas.

Con eso, más la prioridad dada a la accesibilidad WCAG y a la transparencia (código abierto y declaración explícita del uso de IA), se cubre el hueco que las aplicaciones existentes dejan.

3 Análisis y diseño

Este capítulo describe el análisis de requisitos y el diseño técnico de EcoAlerta: metodología, actores, requisitos funcionales y no funcionales, casos de uso, arquitectura, modelo de datos, diseño de la API REST y diseño preliminar de la interfaz. Es la base sobre la que se ha construido la implementación del Capítulo 4.

3.1 Metodología de desarrollo

El desarrollo se ha organizado en seis *sprints* de una semana, con enfoque iterativo e incremental. La razón es práctica: los requisitos cambian con cada tutoría, conviene tener un producto funcional pronto para recibir realimentación, y el calendario disponible es ajustado.

Sobre los *sprints* se aplica **Spec-Driven Development** (SpecKit) (GitHub, [2024](#)), con cuatro fases por iteración:

1. **Specify**: se redacta la especificación funcional del *sprint* y los criterios de aceptación.
2. **Plan**: se descompone en tareas concretas con dependencias.
3. **Implement**: se ejecutan las tareas con asistencia del agente de IA en Google Antigravity.
4. **Verify**: pruebas unitarias, de integración y E2E, y validación de los criterios.

SDD se aplica por una razón específica: como se usa asistencia de IA (sección 1.3), se necesita una especificación previa que sirva tanto de guía al agente como de referencia para revisar lo que produce. El documento maestro `EcoAlerta_Development_Spec_v1.md` vive en el repositorio como fuente de verdad. Se han tenido tutorías con el tutor cada dos semanas; las actas están en `docs/actas-tutorias/`.

3.2 Actores del sistema

Se especifican a continuación, cinco actores con sus respectivos accesos:

Tabla 3.1: Actores del sistema.

Actor	Descripción	Autenticación
Ciudadano anónimo	Consulta el mapa y el detalle de incidencias. No puede crear, votar ni comentar.	Ninguna
Ciudadano registrado	Crea incidencias, vota, comenta y sigue reportes. Tiene perfil social.	JWT
Administrador	Valida, asigna a entidades y cambia estados. Accede al <i>dashboard</i> .	JWT (rol admin)
Entidad responsable	Ayuntamiento, SEPRONA, bomberos, policía local u ONG. Recibe asignaciones.	JWT (rol entity)
Sistema	Notificaciones automáticas, escalado por <i>timeout</i> y <i>audit log</i> .	Interno

La separación entre anónimo y registrado es deliberada: el mapa público funciona como herramienta de transparencia cívica abierta a cualquiera, y al mismo tiempo las acciones con responsabilidad (crear, votar, comentar) requieren cuenta para que queden trazadas.

3.3 Requisitos

Se han definido 46 requisitos funcionales, agrupados en cinco módulos, y 12 requisitos no funcionales. La prioridad funcional es *Alta* (sistema mínimo viable), *Media* (importante para experiencia) o *Baja* (deseable).

3.3.1 Requisitos funcionales

Tabla 3.2: Requisitos funcionales (RF-01 a RF-46).

ID	Pri.	Descripción	Actor
<i>Módulo de autenticación</i>			
RF-01	Alta	Registro con email y contraseña.	Ciudadano
RF-02	Alta	Login con tokens JWT <i>access</i> y <i>refresh</i> .	Todos
RF-03	Alta	Acceso anónimo en lectura (mapa y listado).	Anónimo
RF-04	Alta	Roles: citizen , admin , entity , moderator .	Sistema

ID	Pri.	Descripción	Actor
RF-05	Baja	Recuperación de contraseña por email (limitada por SMTP).	Ciudadano
RF-06	Media	Edición de perfil.	Ciudadano
RF-07	Media	Perfil público con incidencias del usuario.	Ciudadano
Módulo de incidencias			
RF-10	Alta	Creación con título, descripción, categoría, fotos y GPS.	Ciudadano reg.
RF-11	Alta	Captura automática de la ubicación GPS.	Sistema
RF-12	Alta	Selección manual en el mapa si no hay GPS.	Ciudadano
RF-13	Alta	Clasificación por categoría (12 predefinidas).	Sistema
RF-14	Alta	Severidad: leve, moderado, grave, crítico.	Ciud./Admin
RF-15	Alta	1–5 fotos por incidencia, máx. 10 MB cada una.	Ciudadano
RF-16	Media	<i>Thumbnails</i> automáticos.	Sistema
RF-17	Media	Edición/eliminación solo si status = pending .	Ciudadano
RF-18	Media	Historial de cambios en incident_status_history .	Sistema
RF-19	Baja	Borradores con capacidad <i>offline</i> (PWA).	Ciudadano
Módulo de mapa interactivo			
RF-20	Alta	Mapa con todas las incidencias geolocalizadas.	Todos
RF-21	Alta	Agrupación en <i>clusters</i> al disminuir el zoom.	Sistema
RF-22	Alta	Filtros por categoría, severidad, estado, fecha y distancia.	Todos
RF-23	Alta	Marcadores coloreados por severidad.	Sistema
RF-24	Media	Búsqueda por dirección (<i>geocoding</i>).	Todos
RF-25	Alta	Detalle al pulsar el marcador.	Todos
RF-26	Media	Búsquedas por radio (ST_DWithin).	Sistema
Módulo de administración y delegación			
RF-30	Alta	Panel admin con <i>dashboard</i> de estadísticas.	Admin
RF-31	Alta	Cambio de estado: pending → validated → in_progress → resolved/rejected.	Admin
RF-32	Alta	Asignación a entidades responsables.	Admin
RF-33	Alta	Notificación a la entidad y actualización de estado.	Entidad
RF-34	Media	Escalado automático por <i>timeout</i> configurable.	Sistema
RF-35	Media	Estadísticas por categoría, estado, zona, tiempo de resolución.	Admin
RF-36	Media	Gestión de categorías y entidades.	Admin

ID	Pri.	Descripción	Actor
RF-37	Baja	Marcado de duplicadas con vinculación.	Admin
Módulo social y notificaciones			
RF-40	Alta	Votación de incidencias.	Ciudadano
RF-41	Alta	Comentarios con distintivo de oficiales.	Ciud./Ad-min/Ent.
RF-42	Alta	Notificación al autor y seguidores en cambio de estado.	Sistema
RF-43	Media	Seguimiento de incidencias.	Ciudadano
RF-44	Media	Perfil de usuario con incidencias, votos y estadísticas.	Ciudadano
RF-45	Media	Notificaciones <i>push</i> con Service Worker.	Sistema
RF-46	Baja	Notificaciones por email para cambios importantes.	Sistema

3.3.2 Requisitos no funcionales

Los RNF son atributos de calidad medibles. Cada uno se valida con métricas en el Capítulo 5.

Tabla 3.3: Requisitos no funcionales.

ID	Categoría	Descripción
RNF-01	Rendimiento	Tiempo de respuesta API < 500 ms para el 95 % de peticiones.
RNF-02	Rendimiento	Soporte de al menos 100 usuarios simultáneos.
RNF-03	Usabilidad	<i>Responsive</i> : 320 px, 768 px, ≥ 1024 px.
RNF-04	Portabilidad	PWA instalable con <code>manifest.json</code> y Service Worker.
RNF-05	Seguridad	Contraseñas con <i>bcrypt</i> (mínimo 10 rondas).
RNF-06	Seguridad	<i>Rate limiting</i> en todos los <i>endpoints</i> .
RNF-07	Seguridad	HTTPS obligatorio en producción.
RNF-08	Calidad	Cobertura de pruebas unitarias > 70 % en <i>backend</i> .
RNF-09	Calidad	Tests E2E para los 3 flujos principales.
RNF-10	Mantenibilidad	Despliegue reproducible con un comando (<code>docker compose up</code>).
RNF-11	Mantenibilidad	Documentación Swagger/OpenAPI.
RNF-12	Accesibilidad	WCAG 2.1 nivel AA.

3.4 Casos de uso

La Figura 3.1 muestra los casos de uso del sistema agrupados por módulo. La notación `<<requiere>>` marca los casos que necesitan autenticación previa, mientras que `<<include>>` marca los que disparan otro caso de forma encadenada (por ejemplo, validar una incidencia siempre incluye notificar al autor). El actor *Sistema* agrupa los procesos sin intervención humana directa: notificaciones tras cambios de estado (RF-42), escalado por *timeout* (RF-34) y registro de eventos en el *audit log* de seguridad.

3.5 Arquitectura del sistema

Arquitectura cliente-servidor de tres capas:

1. **Presentación:** SPA React 18 (Meta Platforms, 2024) con Tailwind CSS (Tailwind Labs, 2024) y Leaflet (Agafonkin, 2024). Instalable como PWA (Google Developers, 2024).
2. **Lógica:** API REST en Node.js 20 (OpenJS Foundation, 2024b) + Express 4 (OpenJS Foundation, 2024a), con controladores, servicios, *middlewares* y validadores. Swagger en `/api/docs`.
3. **Datos:** PostgreSQL 16 (The PostgreSQL Global Development Group, 2024) con PostGIS 3.4 (PostGIS Project Steering Committee, 2024) (tipos geométricos, `ST_DWithin`, índices GIST).

Sobre las tres capas, un *reverse proxy* nginx termina HTTPS, sirve los estáticos del *frontend* y redirige `/api/*` al *backend*. Todo se despliega con Docker Compose (Docker, Inc., 2024), que da reproducibilidad (RNF-10) y simplifica la demo de la defensa.

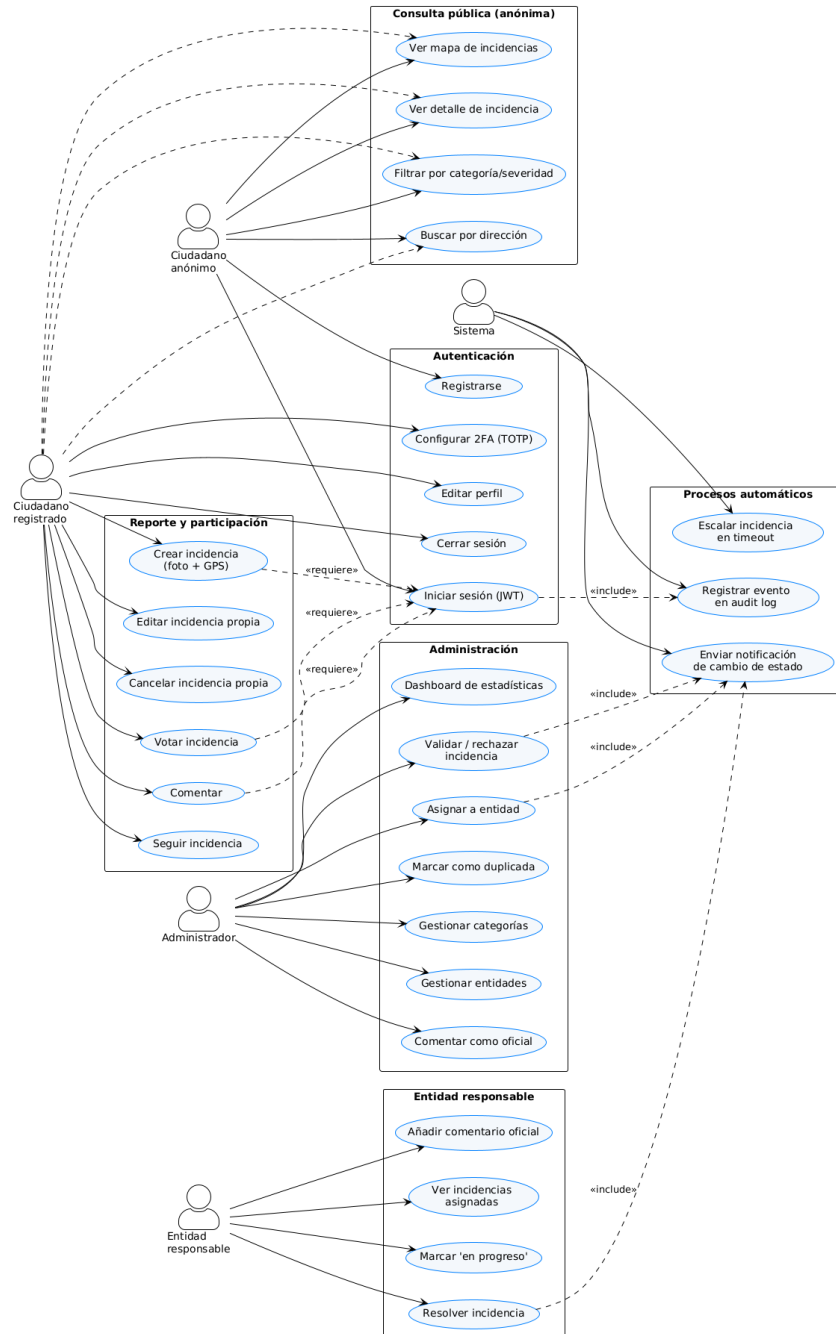


Figura 3.1: Diagrama de casos de uso. Elaboración propia a partir de RF-01–RF-46.

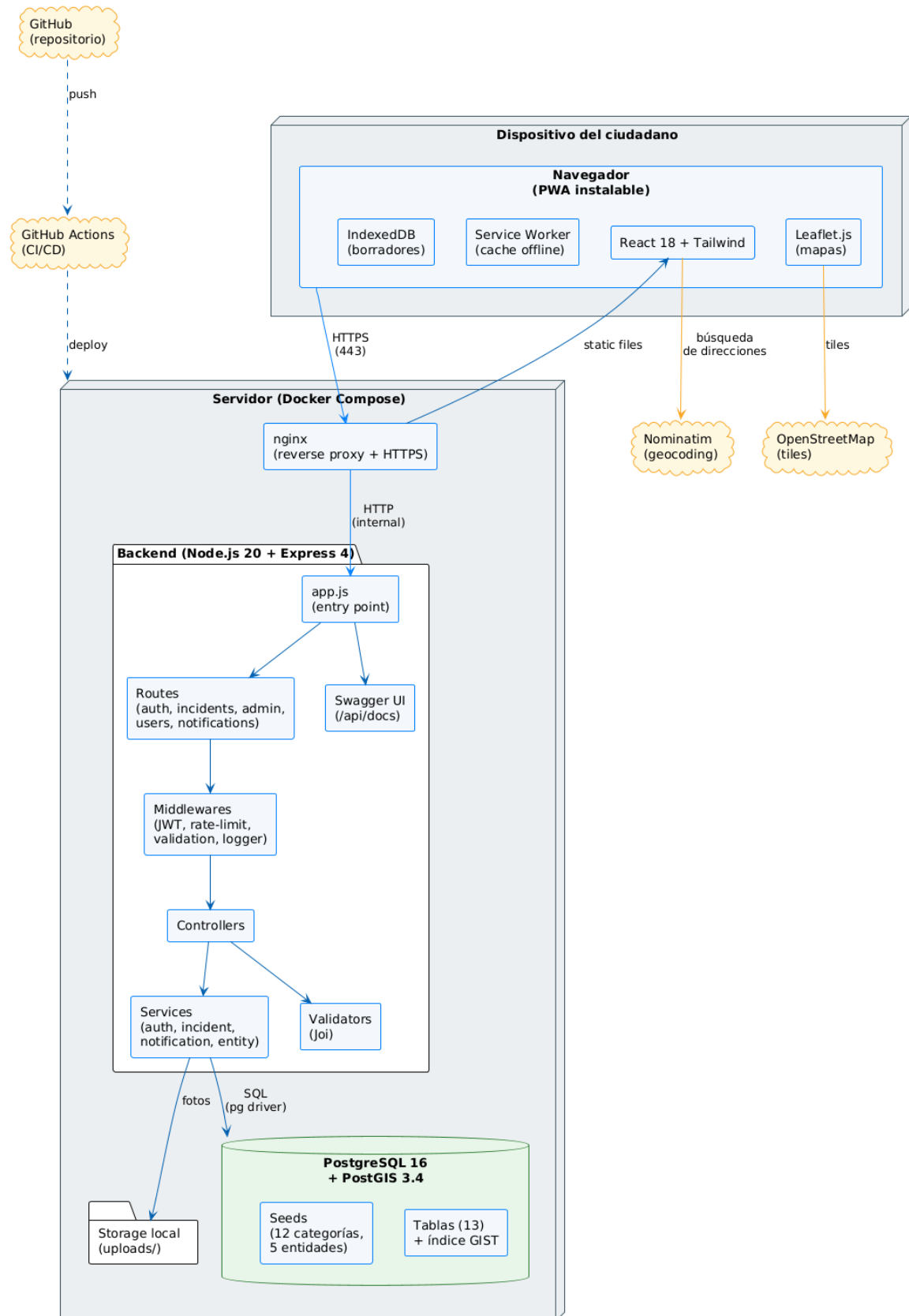


Figura 3.2: Arquitectura de componentes. Elaboración propia.

Servicios externos: *tiles* de OpenStreetMap (OpenStreetMap Contributors, 2024) y *geocoding* contra Nominatim. Ambos gratuitos; evitan dependencias de proveedores comerciales con API de pago, lo que simplifica el modelo de negocio del Anexo 5.

3.5.1 Stack tecnológico

El tutor me dio libertad para elegir el *stack*, con la condición de razonarlo. Es por ello que la Tabla 3.4 resume las tecnologías y la justificación breve de cada elección.

Tabla 3.4: Stack tecnológico.		
Capa	Tecnología	Justificación
Frontend	React 18 + Tailwind	Maduro, integración con Leaflet, <i>responsive</i> sencillo.
Mapas	Leaflet + OSM	<i>Open source</i> , sin API key ni cuota.
PWA	Service Worker	Instalable, caché <i>offline</i> (RF-19, RNF-04).
Backend	Node.js 20 LTS + Express 4	Asíncrono, ecosistema amplio, LTS hasta 2026.
BD	PostgreSQL 16 + PostGIS 3.4	Tipos geométricos, <i>ST_DWithin</i> , índices GIST (RF-26).
Auth	JWT (access+refresh) + bcrypt	Stateless, escalable, RNF-05.
Contenedores	Docker Compose	Despliegue con un comando (RNF-10).
CI/CD	GitHub Actions	Gratuito, <i>lint</i> y tests por <i>push</i> .
Pruebas	Jest + Supertest + Playwright	Pirámide unit/integration/E2E (RNF-08, RNF-09).
Logging	Winston	Estructurado JSON.
API docs	Swagger	Anotaciones en código, sin desincronización (RNF-11).

3.6 Modelo de datos

El modelo tiene 13 entidades, agrupadas en cuatro subsistemas:

- **Identidad y seguridad:** `users`, `user_2fa`, `user_recovery_codes`, `security_audit_log`.
- **Catálogos:** `categories`, `responsible_entities`.
- **Núcleo de incidencias:** `incidents`, `incident_photos`, `incident_comments`, `incident_status_hist`.
- **Interacción social y notificaciones:** `incident_votes`, `incident_follows`, `notifications`.

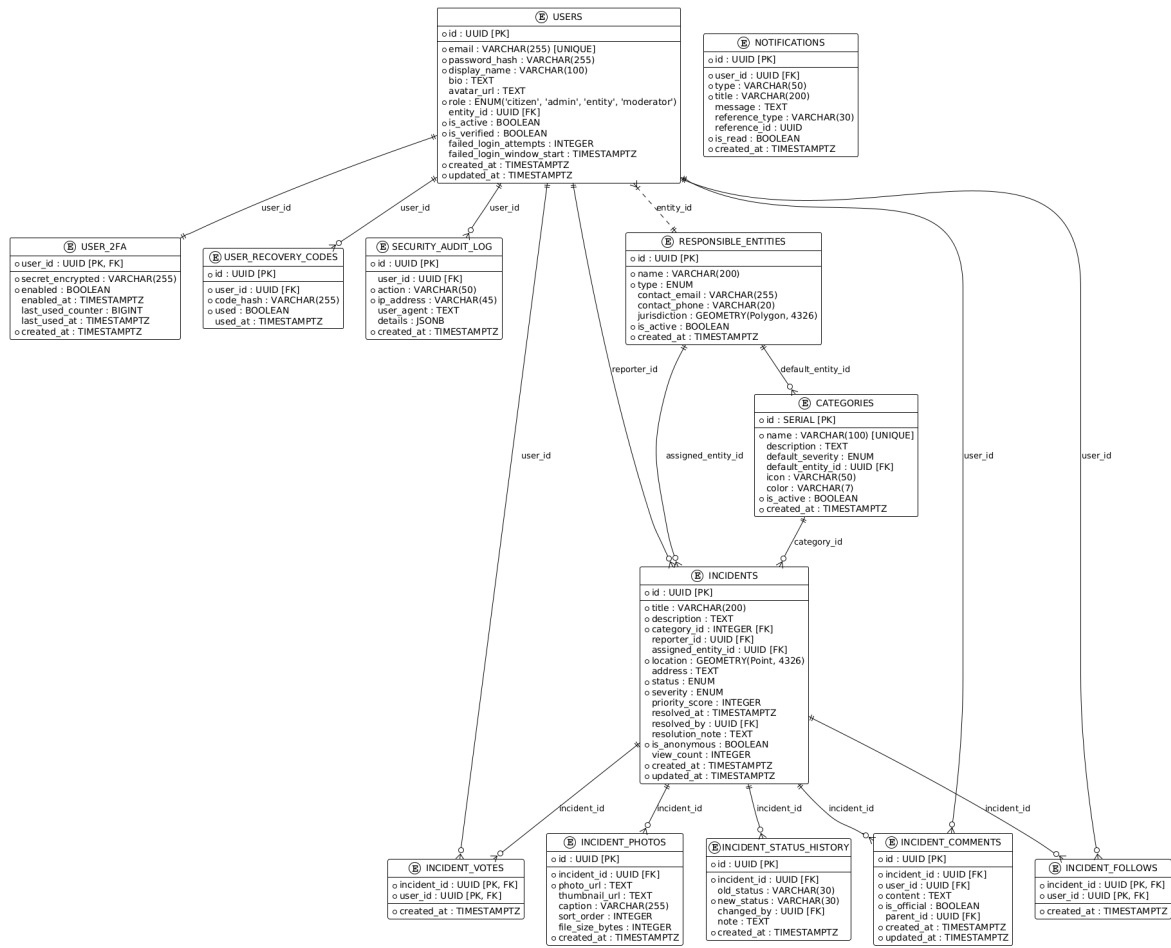


Figura 3.3: Modelo entidad-relación (13 tablas). Elaboración propia.

3.6.1 Entidades principales

users. Información de los usuarios, con `role`, `entity_id`, `is_active` e `is_verified`. Los contadores `failed_login_attempts` y `failed_login_window_start` dan *rate limit* por usuario que refuerza RNF-06.

user_2fa, user_recovery_codes. Añadidas en el Sprint 6. Habilitan 2FA TOTP y guardan códigos de recuperación cifrados.

security_audit_log. Eventos de seguridad (login OK/KO, cambios de rol y contraseña) con IP, *user agent* y JSONB. No se borra: es el *audit trail* del sistema.

responsible_entities. Catálogo de organismos asignables. `jurisdiction` es GEOMETRY(Polygon, 4326) para asignación geoespacial automática a futuro.

categories. Las 12 categorías predefinidas, con severidad por defecto y entidad responsable sugerida. Reduce fricción al reportar.

incidents. Entidad central. `location` es `GEOMETRY(Point, 4326)` con índice GIST para consultas geoespaciales (RF-26). `priority_score` se deriva de severidad y votos para ordenar el *dashboard* (RF-35).

incident_photos, incident_comments, incident_votes, incident_follows. Aquí caen las relaciones que cuelgan de cada incidencia. Las fotos y los comentarios son 1:N (una incidencia tiene varios, cada uno pertenece a una sola). Los votos y los seguimientos asocian usuario con incidencia y son N:M. Como un usuario solo puede votar una vez la misma incidencia, y solo puede seguirla una vez, modelo esa unicidad con clave primaria compuesta (`incident_id, user_id`). La alternativa habría sido una clave `id` sintética más un `UNIQUE` sobre las dos columnas, pero terminaba teniendo dos índices haciendo el mismo trabajo.

incident_status_history. Historial inmutable de transiciones (RF-18). Cada cambio inserta una fila con estado anterior, nuevo, usuario y nota opcional.

notifications. Por usuario. `reference_type` y `reference_id` actúan como *soft foreign key* polimórfica.

3.6.2 Rendimiento geoespacial

Las consultas más frecuentes son del tipo «incidencias en un radio de X km». Las resuelvo con `ST_DWithin` sobre un índice GIST:

 SQL

```
1 CREATE INDEX idx_incidents_location ON incidents USING GIST (location);
2
3 SELECT id, title, severity FROM incidents
4 WHERE ST_DWithin(location::geography,
5                  ST_MakePoint(:lng, :lat)::geography,
6                  :radius_meters);
```

Código 3.2: Índice GIST y consulta geoespacial por radio sobre `incidents`. Elaboración propia.

La proyección a `geography` antes de `ST_DWithin` hace que la distancia se evalúe en metros sobre la superficie terrestre, no en grados sobre el plano cartesiano (que daría resultados incorrectos fuera del ecuador).

3.7 Flujo de estados de una incidencia

Cada incidencia pasa por un ciclo de vida acotado, implementado como `ENUM` en PostgreSQL.

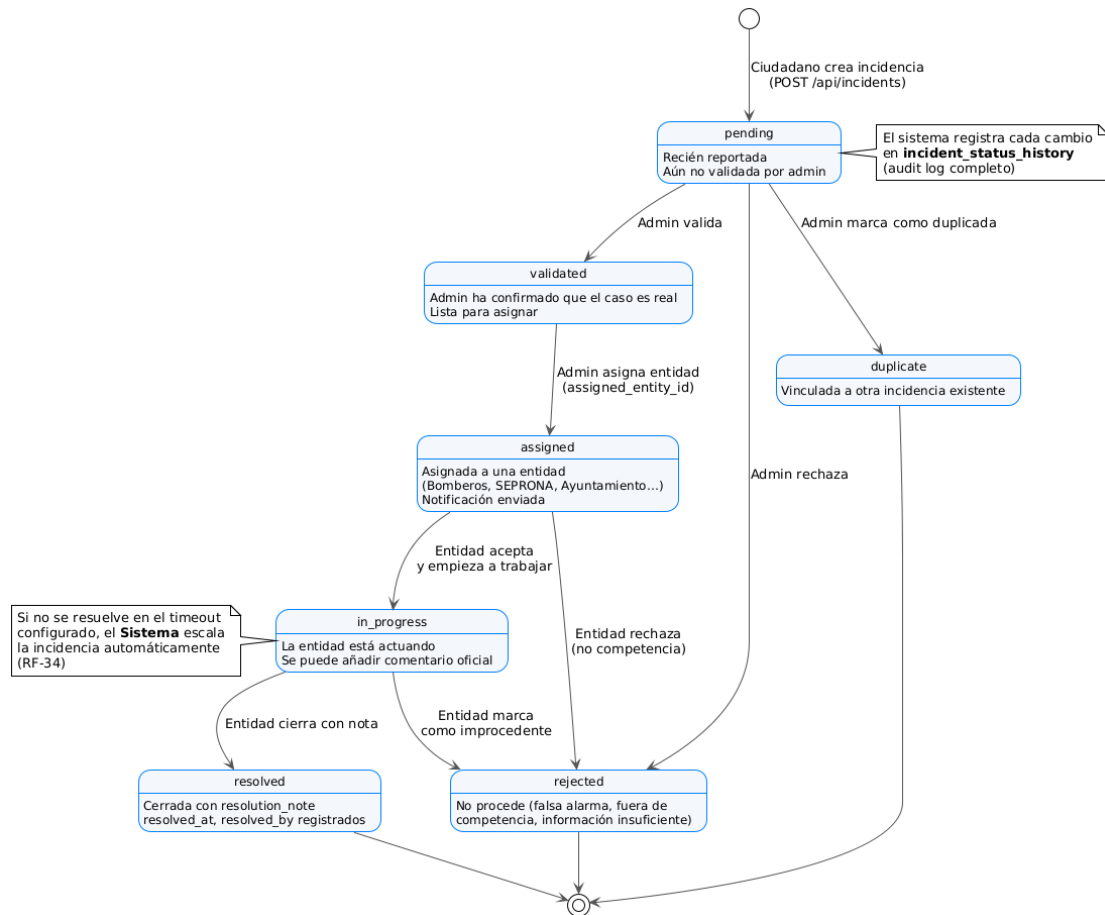


Figura 3.4: Diagrama de estados de la incidencia. Cada transición se guarda en `incident_status_history`. Elaboración propia.

El estado inicial es `pending`. Desde ahí, el admin valida, rechaza o marca como duplicada. Tras validar, asigna a una entidad y la incidencia pasa a `assigned` (con notificación automática al autor, RF-42). La entidad confirma el inicio del trabajo (`in_progress`) y al actuar la cierra como `resolved` con nota obligatoria. Como excepción, una entidad puede rechazar si no es de su competencia. El RF-34 contempla escalado automático por *timeout* si una incidencia en `in_progress` no avanza.

3.8 Diseño de la API REST

La API expone 35 *endpoints* bajo `/api/v1`, organizados en seis recursos. Documentación interactiva Swagger en `/api/docs`.

Tabla 3.5: Endpoints principales.

Método	Ruta	Descripción
<i>Auth</i>		
POST	/auth/register	Registro.
POST	/auth/login	Login (<i>access+refresh</i>).
POST	/auth/refresh	Renovar <i>access token</i> .
GET/PUT	/auth/me	Perfil del usuario autenticado.
<i>Incidents</i>		
GET	/incidents	Listado paginado con filtros.
GET	/incidents/:id	Detalle con fotos y comentarios.
POST	/incidents	Creación (<i>multipart</i>).
PUT/DELETE	/incidents/:id	Edición/baja (autor o admin).
GET	/incidents/nearby	Geoespacial (<i>lat, lng, radius</i>).
GET	/incidents/:id/history	Historial de estados.
POST	/incidents/:id/vote	Votar.
POST	/incidents/:id/follow	Seguir.
POST	/incidents/:id/comments	Comentar.
<i>Admin</i>		
GET	/admin/dashboard	Estadísticas.
GET/PUT	/admin/incidents	Listado y cambios de estado/asignación.
GET/PUT	/admin/users	Gestión de usuarios y roles.
GET/POST/PUT	/admin/entities	CRUD de entidades responsables.
<i>Users / Notifications / Health</i>		
GET	/users/:id/*	Perfil público, incidencias, estadísticas.
GET/PUT	/notifications	Listar y marcar como leídas.
GET	/health	<i>Health check</i> .

La autenticación usa el esquema **Bearer** sobre JWT (Jones et al., 2015). *Access token* de 15 minutos, *refresh token* de 7 días con rotación en cada uso. El *middleware* verifica firma y

vigencia antes de poblar `req.user`.

3.9 Diseño de la interfaz de usuario

Hay tres ideas que han condicionado cómo está pensada la interfaz:

1. **Mobile-first.** El reporte ocurre sobre el terreno, casi siempre desde el móvil. Por eso diseño primero la versión pequeña y dejo que los *breakpoints* de Tailwind adapten el resto a pantallas mayores (RNF-03).
2. **Mapa como eje.** La pantalla de entrada es el mapa, no una lista. Era importante para mí que se viera de un vistazo la densidad y la distribución del problema, porque encaja con la naturaleza cívica del producto.
3. **Accesibilidad.** Contrastes comprobados, navegación por teclado funcionando y etiquetas ARIA en los controles que lo necesitan (RNF-12).

Las pantallas clave son: mapa principal, formulario de reporte, detalle de incidencia, panel admin y perfil de usuario. La Figura 3.5 muestra la pantalla principal en escritorio, donde el mapa ocupa todo el viewport y los filtros se despliegan en un panel lateral. El detalle de las pantallas restantes se incluye en el Capítulo 4 (sección 4.7) y en el manual del Anexo 6.4.

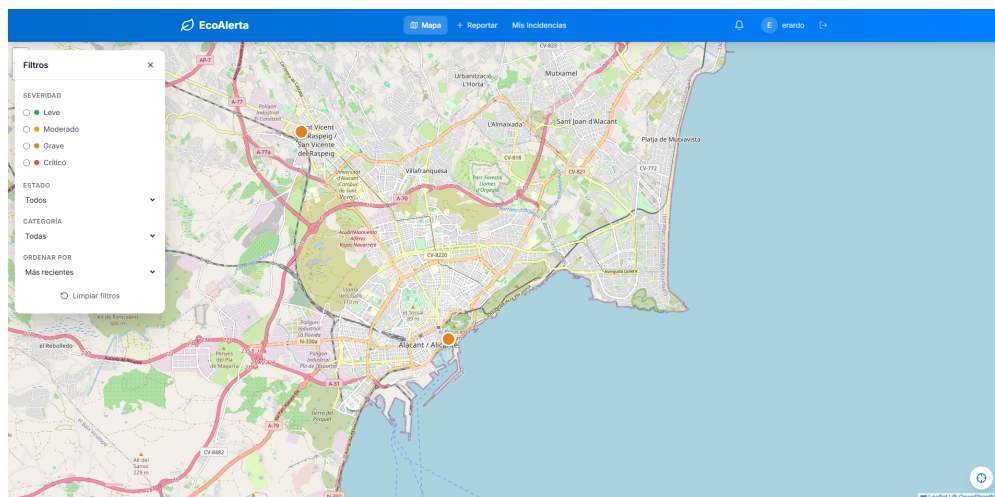


Figura 3.5: Pantalla principal de EcoAlerta: mapa centrado en Alicante con marcadores coloreados por severidad y panel lateral de filtros. Captura propia.

4 Implementación

Este capítulo se basa en documentar el desarrollo. Describe la estructura del repositorio, el entorno, los seis sprints (con las decisiones técnicas más relevantes de cada uno), algunos extractos de código y el *pipeline* de integración continua.

4.1 Entorno y herramientas

Se ha trabajado en Windows 11 con PowerShell. Las herramientas usadas son:

- **IDE:** Google Antigravity (*fork* de VS Code con agentes de IA).
- **Git** con convención *Conventional Commits*.
- **Docker Desktop** con Docker Compose v2.
- **Node.js 20 LTS** y **PostgreSQL 16 + PostGIS 3.4** (en contenedor).
- **Pruebas:** Jest 29 + Supertest 7 (*backend*); Playwright 1.38 (E2E, vía MCP de Antigravity).
- **Bibliografía:** Zotero exportando a BibLaTeX.
- **Documentación API:** Swagger UI en `/api/docs`.
- **MCPs:** GitHub, Context7, Playwright, Figma.

Se eligió Antigravity por su integración nativa con agentes (que encaja con el flujo SDD descrito en la sección 3.1) y la compatibilidad con el ecosistema MCP, que permite ejecutar Playwright sin salir del editor.

4.2 Estructura del repositorio

```
tfg-medioambiente-ua/  
|-- .github/workflows/      # CI/CD GitHub Actions  
|-- database/               # init.sql, seeds  
|-- docs/                   # Documentacion del TFG  
|   |-- memoria/           # Fuentes LaTeX  
|   |-- diagramas/         # Fuentes PlantUML
```

```
|  |-- sprints/                # Specs SDD por sprint
|-- nginx/                    # Reverse proxy
|-- src/
|  |-- backend/               # API Node.js + Express
|  `-- frontend/              # SPA React + Tailwind + PWA
|-- docker-compose.dev.yml
|-- docker-compose.yml
|-- Makefile
`-- EcoAlerta_Development_Spec_v1.md
```

El *backend* sigue la organización *routes* → *controllers* → *services* típica de Express: las rutas definen *endpoints*, los controladores orquestan petición/respuesta, y los servicios encapsulan la lógica y las consultas. Los validadores usan *express-validator*. El *frontend* se organiza por *feature*: *components/*, *pages/*, *context/*, *hooks/*, *services/*, *utils/*.

4.3 Desarrollo por sprints

El desarrollo se organizó en seis *sprints* de una semana. La Tabla 4.1 resume cada uno; mientras que abajo se amplían los puntos técnicos más relevantes.

Tabla 4.1: Resumen de los seis sprints.

S.	Tema	Entregables
1	Auth + BD	Esquema PostGIS, JWT dual, roles, <i>middlewares</i> de seguridad
2	CRUD incidencias	Creación con foto y GPS, validaciones, <i>thumbnails</i> con Sharp
3	Mapa interactivo	Leaflet + <i>clusters</i> , filtros, búsqueda con <i>ST_DWithin</i>
4	Admin + delegación	Dashboard, cambio de estado, asignación, notificaciones
5	Testing + PWA	Jest, Playwright, Service Worker, manifest, Swagger
6	<i>Security hardening</i>	2FA TOTP, Turnstile, <i>audit log</i> , <i>rate limit</i> por usuario

4.3.1 Sprint 1: autenticación y BD

La BD se inicializa con un *init.sql* ejecutado por el contenedor en el primer arranque: habilita PostGIS, crea las trece tablas, define los *ENUM* (*user_role*, *incident_status*, *incident_severity*), crea el índice *GIST* sobre *incidents.location* e inserta las 12 categorías y las 5 entidades por defecto.

Para la autenticación se usa JWT con *access token* de 15 minutos y *refresh token* de 7 días. La función que firma ambos:

```
JS JavaScript

1 const generateTokens = (user) => {
2   const payload = { id: user.id, email: user.email, role: user.role };
3   return {
4     accessToken: jwt.sign(payload, process.env.JWT_SECRET, { expiresIn:
      ↪ '15m' }),
5     refreshToken: jwt.sign(payload, process.env.JWT_SECRET, { expiresIn:
      ↪ '7d' }),
6   };
7 };
```

Código 4.2: Generación de *access* y *refresh tokens* JWT. Elaboración propia. Las contraseñas se guardan en `password_hash` con *bcrypt* a 10 rondas (RNF-05). El *middleware authenticate* verifica el token en cada petición protegida y pone el *payload* en `req.user`; *optionalAuth* permite que los *endpoints* de lectura pública funcionen sin token (RF-03). El control por rol es un *factory*:

```
JS JavaScript

1 const requireRole = (roles) => (req, res, next) => {
2   if (!req.user || !roles.includes(req.user.role))
3     return res.status(403).json({ success: false, error: 'Acceso
      ↪ restringido' });
4   next();
5 };
6 const requireAdmin = requireRole(['admin']);
7 const requireEntity = requireRole(['entity']);
```

Código 4.4: *Middleware* de control de acceso por rol. Elaboración propia.

4.3.2 Sprint 2: CRUD de incidencias

Aquí se monta el flujo principal: crear una incidencia con foto y GPS. Hay que destacar tres puntos:

- Subida *multipart* con *multer*, controlando tamaño y tipo MIME antes del controla-

dor.

- **Thumbnails** JPEG (400 px, calidad 80) con **sharp**, generados en el servidor (RF-16).
- **Ubicación** guardada con `ST_SetSRID(ST_MakePoint(lng, lat), 4326)` para fijar el SRID y evitar ambigüedad en consultas posteriores.

En el *frontend*, la página `CreateIncidentPage` es un formulario por pasos: geolocalización automática (con *fallback* a selección manual sobre el mapa), categoría y severidad, fotos con previsualización, descripción. Si no hay conexión, el borrador queda en IndexedDB y se sincroniza cuando vuelve la red.

4.3.3 Sprint 3: mapa interactivo

Se usa **react-leaflet** sobre *tiles* de OpenStreetMap. Los marcadores se colorean por severidad mediante iconos SVG (RF-23) y se agrupan en *clusters* con `leaflet.markercluster` (RF-21). Los filtros se componen dinámicamente y se proyectan en la consulta al *backend*.

La búsqueda por radio es la consulta más característica del sistema:

```
JS JavaScript

1 const getNearbyIncidents = async (lat, lng, radiusKm = 5, filters = {}) => {
2   const radius = radiusKm * 1000;
3   const where = [`ST_DWithin(i.location::geography,
4     ST_SetSRID(ST_MakePoint($1, $2), 4326)::geography, $3)`];
5   const values = [lng, lat, radius];
6   // ... filtros dinamicos por status, categoryId, severity ...
7   const result = await query(`SELECT i.*, ST_Distance(...) AS distance_meters
8     FROM incidents i WHERE ${where.join(' AND ')}
9     ORDER BY distance_meters ASC LIMIT 50`, values);
10  return result.rows;
11 };
```

Código 4.6: Búsqueda de incidencias cercanas con filtros dinámicos y orden por distancia. Elaboración propia.

Tres detalles importantes: la proyección a **geography** hace que la distancia sea en metros sobre la superficie terrestre y no en grados; los votos se agregan en una subconsulta para no contar filas duplicadas; el orden por **distance_meters** aprovecha que la distancia ya se calculó. El *geocoding* por dirección (RF-24) llama a Nominatim con caché en memoria.

4.3.4 Sprint 4: admin y delegación

Páginas `AdminDashboardPage` (con `Recharts`), `AdminIncidentsPage` (tabla con filtros y acciones masivas), `AdminUsersPage` y `AdminEntitiesPage`. La vista para entidades responsables es `EntityDashboardPage`: filtra automáticamente por `assigned_entity_id` del usuario. El cambio de estado (RF-31) es una transición atómica: la función `changeIncidentStatus` valida que la transición sea coherente con el diagrama de la Figura 3.4, escribe en `incident_status_history`, actualiza el campo `status` y dispara una notificación al autor y a los seguidores. Todo dentro de `BEGIN/COMMIT` para asegurar consistencia.

Las notificaciones se generan en el *backend* y se entregan por *polling* cada 30 segundos desde el componente `NotificationBell`. Elegí *polling* sobre `WebSockets` por simplicidad de despliegue y porque la frecuencia es baja. Las notificaciones *push* reales (RF-45) se montaron en el siguiente sprint con el `Service Worker`.

4.3.5 Sprint 5: testing y PWA

Aquí se consolida la calidad. Bateria de pruebas en tres niveles (unitarias con `Jest`, integración con `Supertest`, E2E con `Playwright`). La cobertura final del *backend* se discute en el Capítulo 5. Queda por debajo del 70 %, sobre todo en *branches*, y se recoge como limitación.

PWA: el `manifest.json` declara iconos, nombre, color y `display: standalone`. El `Service Worker` usa *cache-first* para *assets* estáticos y *network-first* con *fallback* a caché para llamadas API. Esto permite consultar el mapa y las incidencias guardadas sin conexión.

Swagger se genera con `swagger-jsdoc` a partir de anotaciones `@swagger` en las rutas y se sirve en `/api/docs` con `swagger-ui-express`. Documentación y código no se desincronizan.

4.3.6 Sprint 6: security hardening

Además de todo ello, se han añadido tres mejoras de seguridad. Las tablas `user_2fa`, `user_recovery_codes` y `security_audit_log` entraron aquí, pasando el modelo de 10 a 13 entidades.

- **2FA TOTP.** Integración de `otpauth` (RFC 6238) y `qrcode`. El secreto se guarda **cifrado** con AES-256 a partir de una clave maestra fuera del código fuente. Los códigos de recuperación, de un solo uso, se guardan *hasheados*.
 - **Turnstile** (*captcha* de Cloudflare). Componente `@marsidev/react-turnstile` en el *frontend*; `middleware` `turnstile.middleware.js` valida el token contra la API de Cloudflare en el *backend*.
 - **Audit log.** Cada evento de seguridad (login OK/KO, cambio de rol, activación 2FA) se guarda en `security_audit_log` con IP, *user agent* y metadatos JSONB. No se borra.
-

`audit.service.js` centraliza la escritura.

Se ha añadido también *rate limiting* por usuario con los contadores `failed_login_attempts` y `failed_login_window_start` en `users`, complementando el de `express-rate-limit` por IP. La doble protección frena fuerza bruta aunque el atacante rote IPs.

4.4 Frontend: notas adicionales

La aplicación usa `react-router-dom` v6 con rutas públicas (mapa, detalle, login, registro) y rutas protegidas por rol. Las protegidas se envuelven en un `ProtectedRoute` que mira el contexto y redirige a `/login` si toca. *Layout* con `Navbar` en escritorio y `MobileBottomNav` en móvil.

El estado de autenticación vive en `AuthContext`, que expone `login`, `logout`, `refresh` y `user`. Los *tokens* van en `localStorage`. Un interceptor de `Axios` detecta 401 por expiración y renueva el *access token* con el *refresh* antes de reintentar la petición. El *hook* `useAuth` es el punto de acceso al contexto.

Componentes reutilizables clave: `SeverityBadge`, `StatusBadge`, `IncidentMarker`, `MapFilters`, `NotificationBell`, `TwoFactorSetupModal`, `TwoFactorVerifyForm`.

4.5 Despliegue con Docker Compose

`docker-compose.yml` define cinco componentes: `db` (PostgreSQL 16 + PostGIS 3.4 con `init.sql` y volumen `pgdata`), `backend` (Node 20 con Express en el puerto 5000), `frontend` (build de la SPA), `nginx` (*reverse proxy* HTTPS y servidor de estáticos), y el volumen `pgdata`. `docker compose up -d` levanta el sistema completo (RNF-10).

El `Makefile` ofrece atajos: `make dev`, `make test`, `make lint`, `make prod`, `make db-reset`.

4.6 Integración continua

`.github/workflows/ci.yml` se dispara en cada *push* y *pull request*. Tres etapas:

1. **Lint** con ESLint sobre *backend* y *frontend*.
2. **Test** con Jest del *backend* y reporte de cobertura archivado como artefacto.
3. **Build** de la imagen de producción del *frontend*.

El historial de *commits* sigue *Conventional Commits*, lo que permite generar el `CHANGELOG.md` automáticamente.

4.7 Capturas de pantalla

A continuación se muestran las pantallas más representativas de la aplicación en funcionamiento, ordenadas según el flujo típico del ciudadano y la vista del administrador.

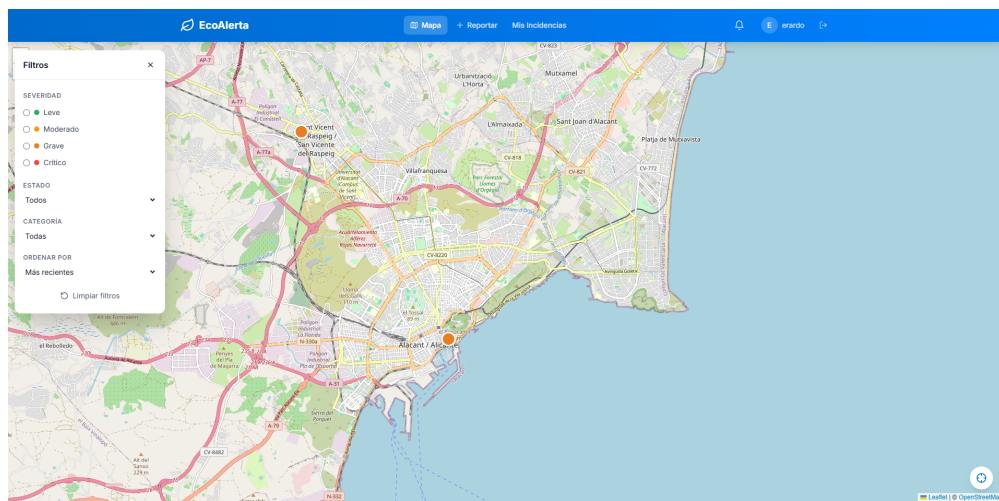


Figura 4.1: Pantalla principal con el mapa de incidencias y el panel lateral de filtros. Captura propia.

The screenshot shows the 'Reportar incidencia' (Report incident) form. The title is 'Reportar incidencia'. The form fields include: 'Título *' (Title) with the placeholder 'Basura tirada en la calle'; 'Descripción *' (Description) with the placeholder 'La calle Alvarez Quintero N 31'; 'Categoría *' (Category) with a dropdown menu showing 'Residuos abandonados'; 'Severidad *' (Severity) with radio buttons for 'Leve', 'Moderado', 'Grave' (selected), and 'Crítico'; 'Ubicación *' (Location) with a map icon and the text 'Haz click en el mapa'; 'Usar mi ubicación' (Use my location) with a location pin icon; 'Fotos (máx. 5)' (Photos (max. 5)) with a camera icon and the text 'Añadir'; and a checkbox for 'Reportar como anónimo' (Report as anonymous). At the bottom is a blue button labeled 'Enviar reporte' (Send report).

Figura 4.2: Formulario de reporte de una incidencia, con categoría, severidad y ubicación. Captura propia.

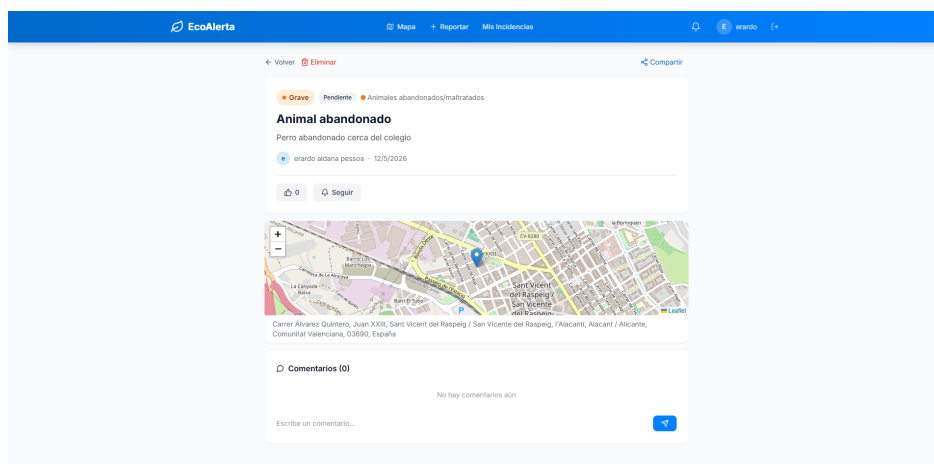


Figura 4.3: Detalle de una incidencia con sus fotos, votos, comentarios y estado. Captura propia.

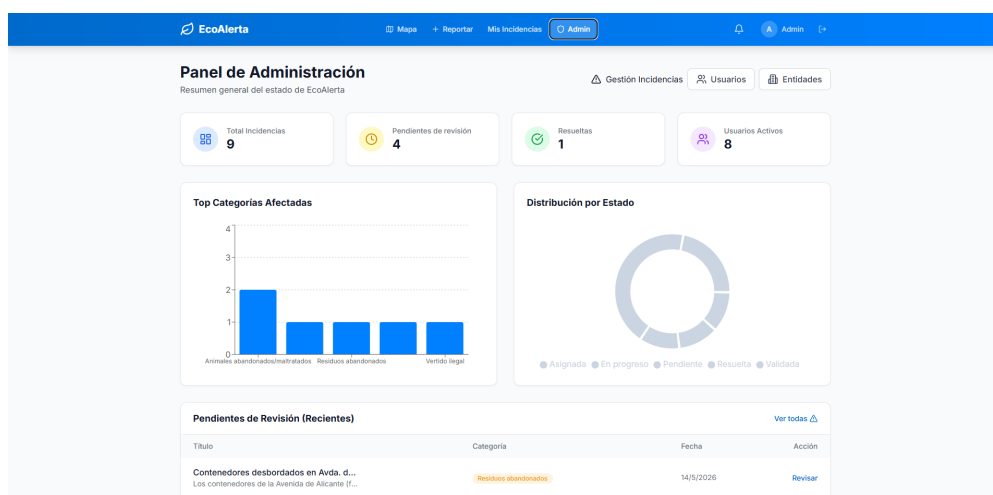


Figura 4.4: Dashboard del administrador con gráficos agregados (Recharts). Captura propia.

4.8 Retos técnicos resueltos

Algunos retos no triviales del proyecto, relevantes para la defensa:

Rendimiento geoespacial

La consulta «incidencias en un radio» se degrada rápido sin índice. `ST_DWithin` con índice GIST hace que PostgreSQL descarte filas antes de calcular distancias, manteniendo latencias por debajo del umbral del RNF-01 incluso con miles de incidencias.

Renovación transparente del *token*

JWT *stateless* simplifica escalar pero complica invalidar. Se resuelve con *access* corto (15 min), *refresh* largo (7 días) con rotación en cada uso, y un interceptor Axios que renueva el *access* sin que el usuario vea un 401 espurio.

Inyección SQL

Todas las consultas usan parámetros posicionales (\$1, \$2...) del *driver pg*, nunca concatenación. En `getNearbyIncidents`, donde el número de filtros varía, los valores se acumulan en un *array* y se pasan al *driver* sin entrar al *parser SQL* como cadena.

Cifrado de secretos 2FA

Un secreto TOTP en claro en BD sería como una contraseña débil ante un *leak*. Lo cifro con AES-256 usando una clave maestra en variable de entorno, así un volcado de BD no basta para comprometer el segundo factor. Los códigos de recuperación van *hasheados* porque nunca se han de recuperar en claro.

Offline con sincronización diferida

El uso típico ocurre en zonas con cobertura inestable (rural, parajes naturales). El Service Worker cachea la UI y las llamadas a `/incidents/nearby`; los borradores van a IndexedDB y se sincronizan al recuperar conexión. Activación automática en navegadores con *Background Sync*.

4.9 Asistencia de IA en la implementación

Tal y como se ha mencionado al inicio del trabajo, en la sección 1.3, parte importante del código se escribió con asistencia del agente de Antigravity siguiendo SDD. El flujo en cada sprint fue:

1. Redacto la especificación del sprint en `docs/sprints/sprint-N.md`: requisitos a cubrir, criterios de aceptación, esquema de pruebas.
 2. El agente genera código a partir de la especificación, con acceso *read-only* al resto del repositorio como contexto.
 3. Se revisa, se ejecutan pruebas locales, se depura y se ajusta manualmente lo que el agente hace mal.
 4. *Commit* en *Conventional Commits* y `CHANGELOG.md` actualizado.
-

Las intervenciones manuales más relevantes ocurrieron en la consulta geoespacial (el agente proponía `ST_Distance` con `WHERE`, ineficiente; se cambió a `ST_DWithin` tras revisar la documentación de PostGIS), en la transacción del cambio de estado (añadí `BEGIN/COMMIT` que el agente había omitido) y en la configuración del Service Worker (donde la estrategia inicial rompía las peticiones autenticadas). En esos puntos la IA propone soluciones que pasan los tests pero no son las óptimas; ahí entra la revisión humana.

5 Evaluación

Una vez cerrado el desarrollo tocaba comprobar que lo construido realmente funcionaba. Me apoyo en pruebas automatizadas a varios niveles (Jest y Supertest en el *backend*, Playwright para los flujos completos), en la verificación punto por punto de los requisitos del Capítulo 3 y en un protocolo de evaluación con usuarios reales que dejo planificado como trabajo futuro inmediato.

5.1 Pruebas automatizadas del *backend*

Las pruebas del *backend* corren sobre Jest 29 y Supertest 7. Las he repartido en tres bloques según lo que cubre cada uno.

El primero contiene las pruebas unitarias. Aíslan cada controlador o *middleware* y mockean lo que necesitan a su alrededor: base de datos, servicios externos, dependencias auxiliares. Son el grueso de la suite porque salen baratas de mantener y se ejecutan rápido. Aquí entran `auth.controller.test.js`, `auth.middleware.test.js`, `auth.service.test.js`, `admin.controller.test.js`, `admin.service.test.js`, `error.middleware.test.js`, `incident.controller.test.js`, `incident.service.test.js`, `notification.service.test.js`, `photo.service.test.js`, `upload.middleware.test.js`, `routes.test.js` y `geocoding.service.test.js`.

El segundo bloque cubre los servicios añadidos en el Sprint 6, todos relacionados con seguridad: `crypto.service.test.js`, `qr.service.test.js`, `turnstile.service.test.js` y `twofa.service.test.js`. Aquí he puesto más cuidado del habitual, porque un fallo en el cifrado o en la generación de códigos TOTP no aparece a simple vista pero sí en una auditoría. El tercer bloque levanta una instancia real de Express y comprueba las rutas de extremo a extremo sin mocks: `auth.routes.test.js`. El patrón es replicable al resto de recursos y queda apuntado como ampliación natural de la suite.

La cobertura final, medida con el reporte HTML de Jest (`src/backend/coverage/lcov-report/index.html`), se muestra en la Tabla 5.1.

Tabla 5.1: Cobertura final del *backend*.

Métrica	Cobertura	Objetivo (RNF-08)	Comentario
<i>Statements</i>	61,52 %	> 70 %	Por encima del 50 % mínimo
<i>Branches</i>	38,91 %	> 70 %	No cumple (limitación)
<i>Functions</i>	62,40 %	> 70 %	Mayoría de controladores
<i>Lines</i>	62,92 %	> 70 %	Lógica de negocio cubierta

La cobertura está por debajo del 70 % que se marcó. La razón es que muchas ramas de manejo de error (*catch* de excepciones específicas, *timeouts* de servicios externos, escenarios de concurrencia) no se prueban explícitamente. Este hecho es discutido como limitación en el próximo capítulo.

5.2 Pruebas *end-to-end* con Playwright

Las pruebas E2E corren con Playwright sobre la aplicación completa levantada con Docker Compose. Cubren los tres flujos principales (RNF-09) y un cuarto de seguridad añadido en el Sprint 6. La Tabla 5.2 resume los *specs*.

Tabla 5.2: Pruebas E2E con Playwright.

Spec	Escenarios
<code>incident-report.spec.js</code>	Registro, login, creación de incidencia con foto y GPS, marcador en el mapa, voto, comentario.
<code>map-navigation.spec.js</code>	Zoom, paneo, filtros por categoría y severidad, búsqueda por dirección, detalle.
<code>admin-management.spec.js</code>	Acceso al <i>dashboard</i> , cambio de estado, asignación a entidad, notificación al autor.
<code>security.spec.js</code>	Activación de 2FA, verificación en <i>login</i> , código de recuperación, límite de intentos.

Las pruebas se ejecutan en Chromium, Firefox y WebKit en paralelo. Un ciclo completo de las cuatro *specs* en los tres navegadores tarda unos cuatro minutos.

5.3 Validación del cumplimiento de requisitos

Resumen del cumplimiento (✓ cumplido, ~ parcial, × fuera de alcance). Los requisitos parciales se deben sobre todo a infraestructura externa (SMTP, certificados, *scheduler*) que no

se ha desplegado en desarrollo.

Tabla 5.3: Resumen del cumplimiento de requisitos funcionales.

ID		Evidencia
RF-01..04	✓	Registro, login JWT, acceso anónimo y roles verificados con tests unitarios e integración.
RF-05	~	Implementado, sin SMTP real para validar end-to-end.
RF-06..07	✓	Edición de perfil y perfil público operativos.
RF-10..18	✓	CRUD de incidencias con foto, GPS, severidad, historial e índice GIST verificado.
RF-19	~	Borradores en IndexedDB OK; <i>Background Sync</i> solo en Chrome.
RF-20..26	✓	Mapa con <i>clustering</i> , filtros, <i>ST_DWithin</i> , <i>geocoding</i> .
RF-30..33	✓	Dashboard, cambios de estado, asignación a entidad y notificaciones.
RF-34	~	Diseñado pero falta el <i>scheduler</i> (trabajo futuro).
RF-35..36	✓	Estadísticas y CRUD de catálogos en panel admin.
RF-37	~	API disponible, UI simplificada.
RF-40..44	✓	Votos, comentarios oficiales, notificaciones, seguimiento, perfil social.
RF-45	~	Service Worker registrado; <i>push</i> requiere VAPID en producción.
RF-46	×	Fuera de alcance (sin SMTP).

Tabla 5.4: Cumplimiento de requisitos no funcionales.

ID		Evidencia
RNF-01	✓	Latencia geoespacial < 300 ms en local.
RNF-02	~	Sin prueba de carga (dimensionado teórico OK).
RNF-03	✓	<i>Responsive</i> verificado en 320, 768 y 1024 px.
RNF-04	✓	PWA instalable.
RNF-05	✓	<i>bcrypt</i> con 10 rondas.
RNF-06	✓	<i>Rate limit</i> por IP y por usuario.
RNF-07	~	nginx preparado, falta certificado real.
RNF-08	~	Cobertura 61–63 % (objetivo 70 %).
RNF-09	✓	4 <i>specs</i> E2E.
RNF-10	✓	<code>docker compose up -d</code> .
RNF-11	✓	Swagger en <code>/api/docs</code> .
RNF-12	~	Contrastes y teclado verificados; auditoría WCAG completa pendiente.

5.4 Pruebas de usabilidad: protocolo y trabajo futuro

La validación de la usabilidad con usuarios reales se ha planificado siguiendo un protocolo ligero pensado para detectar problemas graves de interacción (*think-aloud* sobre las tareas críticas). La realización efectiva con una muestra suficiente queda como trabajo futuro inmediato; se discuten las limitaciones derivadas en el Capítulo 6 y se desarrollan los pasos previstos en la sección 6.3.

Protocolo planificado

1. Reclutamiento de entre tres y cinco personas con perfiles distintos (rango de edad, familiaridad con la tecnología, experiencia previa con aplicaciones cívicas).
2. Presentación breve del objetivo del sistema sin instrucciones detalladas, para observar la interacción espontánea.
3. Tres tareas: consultar una incidencia en el mapa, crear una incidencia con foto y votar una existente. Cada tarea se cronometra y se marca como completada o fallida.
4. Observación sin intervención del facilitador, anotando dificultades, vacilaciones y verbalizaciones del participante.
5. Entrevista breve al cierre con cinco preguntas abiertas sobre claridad, fluidez y aspectos a mejorar.

Indicadores planeados

Se prevén dos clases de indicadores. Los cuantitativos (tiempo medio por tarea, tasa de finalización, número de errores graves) permitirán comparar el rendimiento entre perfiles. Los cualitativos (frases textuales del participante, puntos de fricción identificados, sugerencias) ofrecerán pistas concretas para iteraciones posteriores. La síntesis se organizará en tres bloques: hallazgos positivos comunes a todos los participantes, fricciones detectadas y recomendaciones de mejora.

5.5 Valoración global

Tras la evaluación se destaca lo siguiente:

- Todos los requisitos funcionales de prioridad alta están cumplidos y respaldados por pruebas. Los parciales son consecuencia de no tener desplegada la infraestructura externa.
- Los requisitos no funcionales se cumplen en su mayoría; la cobertura del *backend* es el *gap* más visible (61–63 % frente a 70 %), aunque está por encima del 50 % que se considera mínimo industrial.
- El sistema es reproducible con un comando, lo que satisface OBJ-2 y OBJ-8.
- La pirámide de pruebas tiene la forma adecuada: muchas unitarias, integración moderada, E2E sobre los tres flujos principales más seguridad.

Las limitaciones se discuten con detalle en el capítulo siguiente, junto con las líneas de trabajo futuro.

6 Conclusiones y trabajo futuro

6.1 Logros

Repasando los ocho objetivos planteados en la sección 1.2, siete de ellos están plenamente cumplidos: la arquitectura, el modelo de datos, la API REST documentada, la interfaz PWA, los mecanismos de seguridad del Sprint 6, el análisis del estado del arte y la documentación reproducible. El objetivo 7 (validación con pirámide de pruebas y usuarios reales) queda parcialmente cumplido, ya que la pirámide de pruebas automatizadas (unitarias, integración y E2E) está operativa, pero la validación de usabilidad con usuarios reales sigue pendiente y se desarrolla como línea de trabajo futuro.

Transparencia con el uso de IA

El proyecto es coherente con el contexto en el que se desarrolla. Existen herramientas de asistencia y las he usado, declarándolo y dejando trazas auditables. La sección 1.3 y el Capítulo 4 entran en detalle. Creo que esa actitud es la única razonable a día de hoy.

Calidad cercana a producción

El sistema incluye CI/CD con GitHub Actions, *audit log* de seguridad, documentación API autogenerada, capacidad *offline* y 2FA. Estas funcionalidades dan lugar a un resultado más profesional, aunque no eran estrictamente necesarias para cumplir el enunciado.

Reusabilidad

El modelo de datos, el flujo de estados y el patrón de delegación a entidades pueden trasladarse con pocos cambios a dominios cercanos: seguridad ciudadana, patrimonio, salud pública. El diseño no se queda en lo medioambiental.

6.2 Limitaciones

Reconocer lo que no está bien terminado es parte del trabajo:

Cobertura de pruebas

La cobertura de *branches* (38,91 %) está claramente por debajo del 70 % que fijé en RNF-08. Muchas ramas de error no se prueban. Es un trabajo mecánico al que no se le ha dado tiempo

para cerrar.

Sin SMTP real

La recuperación de contraseña por email (RF-05) y las notificaciones por email (RF-46) están programadas pero no se han probado contra un servidor real. Funcionarán al configurar las variables de entorno en producción.

Escalado por *timeout* no programado

El RF-34 (escalado de incidencias no atendidas) está diseñado pero no hay un *scheduler* corriendo. Falta añadir cron o un *worker* dedicado.

Pruebas de usabilidad pendientes

La validación con usuarios reales sigue el protocolo descrito en la sección 5.4, pero su ejecución completa ha quedado fuera del alcance de esta entrega. Una evaluación sólida requeriría entre 5 y 15 participantes con perfiles diversos y se aborda como trabajo futuro inmediato.

Sin prueba de carga

El RNF-02 (100 usuarios simultáneos) se ha validado por dimensionado teórico, no con *k6* o *Artillery* contra el sistema real.

Moderación social básica

Los comentarios entre usuarios funcionan, pero no hay filtros de lenguaje, reputación ni *shadow banning*. En un despliegue público abierto haría falta.

6.3 Trabajo futuro

Las líneas que veo claras como continuación natural:

Corto plazo. Subir la cobertura por encima del 80 %, desplegar en *cloud* con HTTPS real (Let's Encrypt) y backups automatizados, programar el *scheduler* de RF-34, y pasar pruebas de carga con *k6* para validar RNF-02.

Medio plazo. Una aplicación nativa complementaria con React Native, aprovechando la API REST existente, para mejorar las notificaciones *push* y el acceso a sensores. Asignación automática de incidencias a entidades por intersección geoespacial entre ubicación y jurisdicción (el campo *jurisdiction* ya existe en el modelo). Sistema de reputación del ciudadano basado en histórico. Clasificación automática de la fotografía mediante visión por computador, que sugiera categoría y severidad al crear la incidencia.

Largo plazo. Convenios con ayuntamientos piloto en la provincia de Alicante para validar el producto en producción. Integración con plataformas de gestión municipal (GESCON, Gestiona). Modelo de negocio B2G *freemium* desarrollado a partir del análisis del Anexo 5. Apertura del código con licencia MIT o Apache 2.0 para fomentar replicación en otros municipios.

6.4 Reflexión

El TFG ha permitido juntar en un único producto cosas que vimos en distintas asignaturas del Grado: bases de datos, programación web, arquitectura, ingeniería de requisitos, pruebas, seguridad, gestión de proyectos. Hasta ahora cada una vivía en su silo; aquí se han tenido que coordinar.

En lo metodológico, combinar SDD con agentes de IA en un IDE como Antigravity ha sido un aprendizaje en sí mismo. Lo más probable es que sea así como va a trabajar buena parte de la industria de aquí a unos años. La conclusión que se obtiene es que la diferencia no la marca delegar, sino dirigir bien: especificar con precisión, revisar con rigor y firmar el resultado.

Por último, se agradece al tutor José Luis Sánchez Romero su disponibilidad y la libertad que me dio para elegir el *stack* y el alcance. Sin esa libertad, EcoAlerta no habría salido como ha salido.

Bibliografía

- Agafonkin, V. (2024). *Leaflet: An Open-Source JavaScript Library for Mobile-Friendly Interactive Maps*. Consultado el 24 de marzo de 2026, desde <https://leafletjs.com/>
- California Academy of Sciences and National Geographic Society. (2024). *iNaturalist: A Community for Naturalists*. Consultado el 24 de marzo de 2026, desde <https://www.inaturalist.org/>
- Docker, Inc. (2024). *Docker: Accelerated Container Application Development*. Consultado el 24 de marzo de 2026, desde <https://www.docker.com/>
- GitHub. (2024). *SpecKit: Toolkit for Spec-Driven Development*. Consultado el 24 de marzo de 2026, desde <https://github.com/github/spec-kit>
- Goodchild, M. F. (2007). Citizens as Sensors: The World of Volunteered Geography. *Geo-Journal*, 69(4), 211-221.
- Google Developers. (2024). *Progressive Web Apps*. Consultado el 24 de marzo de 2026, desde <https://web.dev/progressive-web-apps/>
- Haklay, M. (2013). Citizen Science and Volunteered Geographic Information: Overview and Typology of Participation. *Crowdsourcing Geographic Knowledge*, 105-122.
- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* [RFC 7519]. Consultado el 24 de marzo de 2026, desde <https://datatracker.ietf.org/doc/html/rfc7519>
- Línea Verde Smart City. (2024). *Línea Verde: Comunicación Ciudadana Municipal*. Consultado el 24 de marzo de 2026, desde <https://www.lineaverdesmart.com/>
- Meta Platforms. (2024). *React: A JavaScript Library for Building User Interfaces*. Consultado el 24 de marzo de 2026, desde <https://react.dev/>
- mySociety. (2024). *FixMyStreet: Report Problems in Your Area*. Consultado el 24 de marzo de 2026, desde <https://www.fixmystreet.com/>
- OpenJS Foundation. (2024a). *Express: Fast, Unopinionated, Minimalist Web Framework for Node.js*. Consultado el 24 de marzo de 2026, desde <https://expressjs.com/>
- OpenJS Foundation. (2024b). *Node.js*. Consultado el 24 de marzo de 2026, desde <https://nodejs.org/>
- OpenStreetMap Contributors. (2024). *OpenStreetMap*. Consultado el 24 de marzo de 2026, desde <https://www.openstreetmap.org/>
- PostGIS Project Steering Committee. (2024). *PostGIS: Spatial and Geographic Objects for PostgreSQL*. Consultado el 24 de marzo de 2026, desde <https://postgis.net/>
- Tailwind Labs. (2024). *Tailwind CSS: A Utility-First CSS Framework*. Consultado el 24 de marzo de 2026, desde <https://tailwindcss.com/>

The PostgreSQL Global Development Group. (2024). *PostgreSQL: The World's Most Advanced Open Source Relational Database*. Consultado el 24 de marzo de 2026, desde <https://www.postgresql.org/>

Guía de instalación

Este anexo recoge el proceso para levantar EcoAlerta desde cero en una máquina de desarrollo. Los comandos están probados en Windows 11 con PowerShell, pero son equivalentes en Linux (bash/zsh) y macOS.

Requisitos previos

- **Docker Desktop** 4.30 o superior, con Compose v2.
- **Git** 2.40 o superior.
- Puertos libres: 3000, 5000, 5432, 80, 443.
- 4 GB de RAM libre y 5 GB de disco.
- (Opcional) **Make** para los atajos del Makefile.

Clonado y configuración

```
git clone https://github.com/eap59-ua/tfg-medioambiente-ua.git
cd tfg-medioambiente-ua
Copy-Item .env.example .env          # PowerShell
# o:  cp .env.example .env          # bash/zsh
```

Las variables de entorno imprescindibles para desarrollo local son DB_PASSWORD, JWT_SECRET y MFA_KEY. Las dos últimas conviene generarlas con `openssl rand -base64 48` y `openssl rand -base64 32`. El resto puede quedarse con valores por defecto.

Arranque

Con Make:

```
make dev
```

Sin Make:

```
docker compose -f docker-compose.dev.yml up --build
```

El primer arranque tarda 3–8 minutos (descarga imágenes y construye). Cuando aparezca `Server listening on port 5000`, los servicios están disponibles:

- **Frontend:** <http://localhost:3000>
- **Backend API:** <http://localhost:5000/api/v1>
- **Swagger UI:** <http://localhost:5000/api/docs>
- **PostgreSQL:** `localhost:5432`

Credenciales por defecto

Los *seeds* crean un administrador y cinco entidades responsables:

- **Email:** `admin@ecoalerta.es`
- **Password:** `Admin123!`

Cambiar obligatoriamente en cualquier despliegue real.

Comandos útiles

Comando	Descripción
<code>make dev</code>	Entorno de desarrollo con <i>hot-reload</i> .
<code>make stop</code>	Para los contenedores.
<code>make test</code>	Ejecuta Jest del <i>backend</i> .
<code>make lint</code>	ESLint en <i>backend</i> y <i>frontend</i> .
<code>make rebuild</code>	Reconstruye sin caché.
<code>make logs</code>	Logs de todos los servicios.
<code>make prod</code>	Entorno de producción.

Problemas frecuentes

Puerto 5432 ocupado. Hay PostgreSQL local corriendo. Páralo o cambia el mapeo en `docker-compose.dev.yml`.

BD no inicializada. El contenedor ya existía y `init.sql` no se ejecutó. Borrar volumen: `docker compose down -v && make dev`.

Permisos en Windows. Si Docker se queja de `node_modules`, habilitar el *filesystem sharing* para el disco del repositorio en la configuración de Docker Desktop.

Tokens inválidos al reiniciar. JWT_SECRET no debe cambiar entre reinicios; los tokens emitidos antes dejarían de validar.

Manual de usuario

Esta guía explica cómo se usa EcoAlerta desde cada uno de los perfiles definidos en la sección 3.2. Dividida por rol; cada operación se acompaña de la captura correspondiente.

1 Ciudadano anónimo

Sin registrarse, el ciudadano puede consultar el mapa público.

Consultar el mapa. Al entrar a <http://localhost:3000> aparece el mapa con los marcadores de las incidencias activas, centrado por defecto en la Universidad de Alicante. El color del marcador refleja la severidad (verde leve, amarillo moderado, naranja grave, rojo crítico). La Figura 1 muestra la vista en un dispositivo móvil de 375 px de ancho.

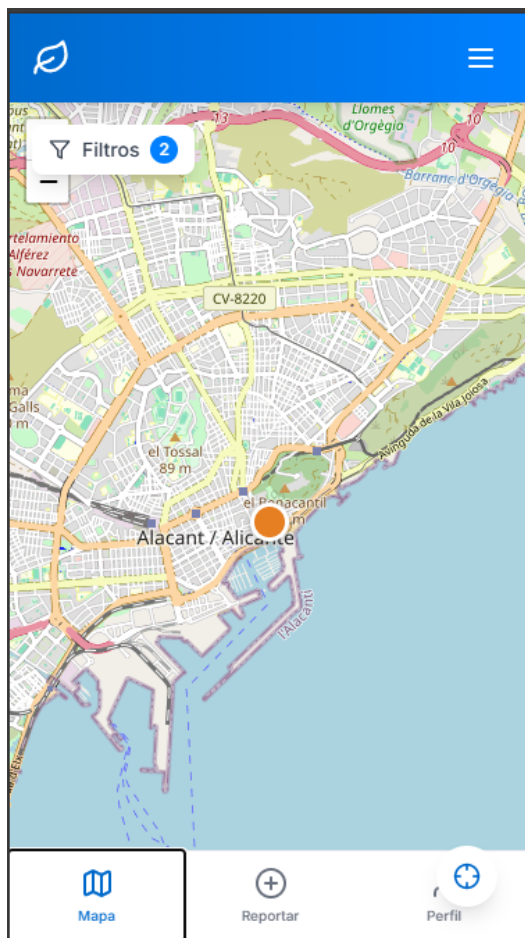


Figura 1: Vista del mapa en dispositivo móvil (*viewport* 375 px). Captura propia.

Filtrar. El botón «Filtros» despliega un panel con categoría, severidad, estado, rango de fechas y distancia desde la posición del usuario. Los filtros se combinan con AND.

Consultar el detalle. Click en un marcador abre un *popup* con resumen y enlace al detalle, donde se ven las fotos, descripción, historial de estados, comentarios (incluidos los oficiales) y número de votos.

2 Ciudadano registrado

Tras registrarse y verificar la cuenta, además de lo anterior:

Registro y login. Email válido, contraseña segura (mínimo 8 caracteres, una mayúscula, un número, un símbolo) y nombre de usuario. La Figura 2 muestra el formulario de registro.

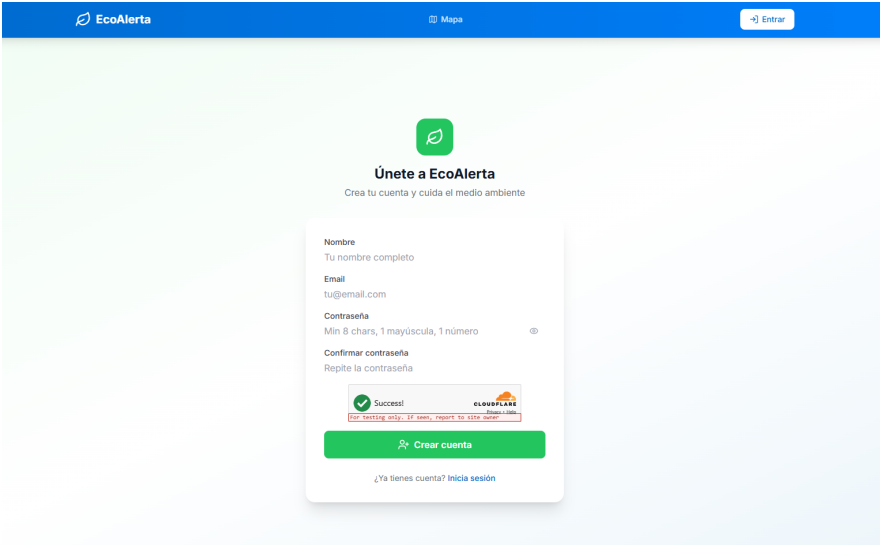
La imagen muestra la interfaz de usuario para el registro en EcoAlerta. En la parte superior hay una barra azul con el logo 'EcoAlerta', un icono de 'Mapa' y un botón 'Entrar'. El fondo es verde claro. En el centro, un recuadro blanco contiene el título 'Únete a EcoAlerta' con un icono de hoja verde, seguido del subtítulo 'Crea tu cuenta y cuida el medio ambiente'. El formulario pide: 'Nombre' (Tu nombre completo), 'Email' (tu@email.com), 'Contraseña' (Min 8 chars, 1 mayúscula, 1 número) y 'Confirmar contraseña' (Repite la contraseña). Hay un botón verde 'Crear cuenta' y un enlace '¿Ya tienes cuenta? Inicia sesión'. Una barra de estado roja indica 'Success!'. En la esquina superior derecha del formulario hay un logo de 'CLOUDFLARE'.

Figura 2: Formulario de registro del ciudadano. Captura propia.

Desde el perfil se puede activar 2FA por TOTP con Google Authenticator, Authy o equivalente. La Figura 3 muestra el modal de activación con el código QR generado por el servidor (el QR de la captura se ha difuminado por seguridad).

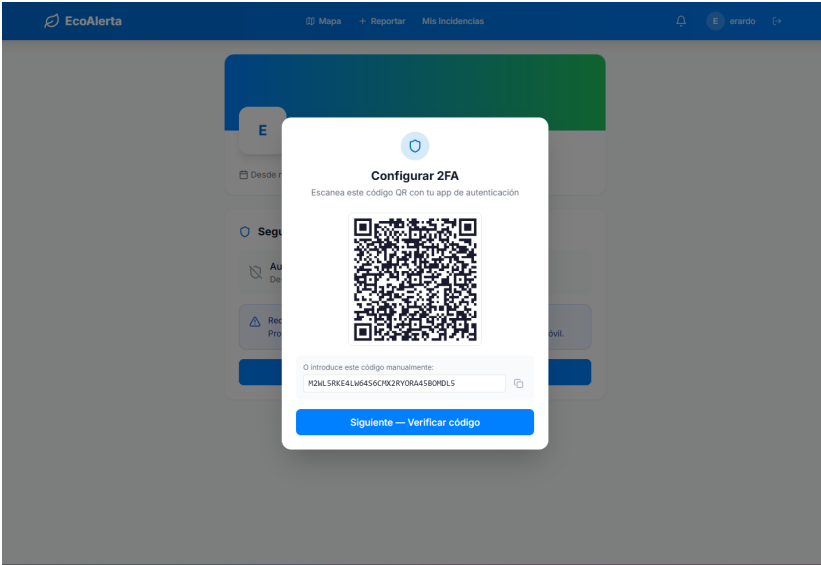
La imagen muestra un modal de configuración de 2FA sobre una interfaz de usuario desenfocada. El modal tiene el título 'Configurar 2FA' y el subtítulo 'Escanee este código QR con tu app de autenticación'. En el centro hay un código QR. Debajo, un campo de texto permite introducir el código manualmente, con el valor 'PQZLSRKE4LW64S6CHXZRYOR445B0YDL5'. En la parte inferior hay un botón azul 'Siguiente — Verificar código'.

Figura 3: Modal de activación del segundo factor (2FA) con código QR. Captura propia.

Crear una incidencia. Botón «Reportar». Pasos:

1. Permitir acceso a la ubicación cuando lo pida el navegador.

2. Verificar o ajustar la posición sobre el mapa.
3. Categoría (12 opciones) y severidad (el sistema sugiere una por defecto).
4. Título y descripción breve.
5. Subir 1–5 fotos (máx. 10 MB cada una).
6. Pulsar «Publicar».

Sin conexión, el borrador queda en IndexedDB y se sincroniza al recuperar la red.

Votar, comentar, seguir. Desde el detalle: el icono de apoyo (un voto por usuario), comentarios libres (los del autor se destacan) y «Seguir» para recibir notificaciones de cambios.

Mi perfil. Listado de mis incidencias, mis votos, mis comentarios y estadísticas. Activación de 2FA y cambio de contraseña.

3 Administrador

Acceso al panel /admin con rol **admin**.

Dashboard. Gráficos de incidencias por categoría, por estado, evolución temporal, distribución por municipio y tiempo medio de resolución. La Figura 4 muestra la pantalla de gestión de incidencias del administrador, con tabla filtrable y acciones masivas.

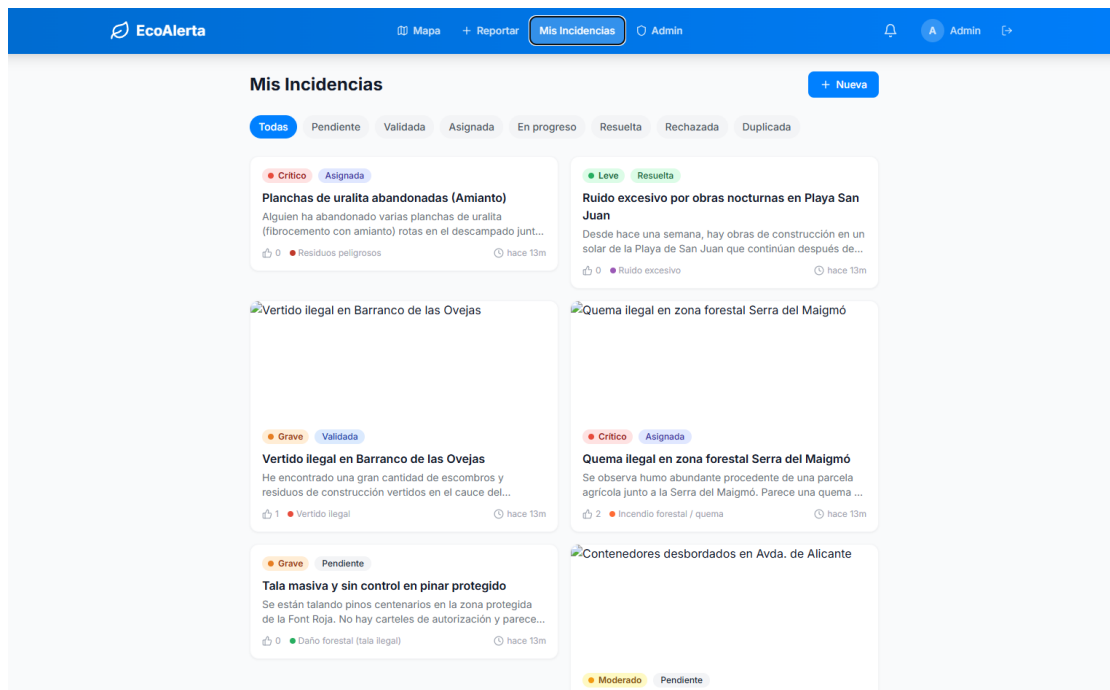


Figura 4: Panel de gestión de incidencias del administrador. Captura propia.

Gestión de incidencias. Tabla con filtros avanzados. Acciones por incidencia: validar/rechazar, asignar a entidad, marcar como duplicada, comentar como oficial, cambio manual de estado.

Usuarios y entidades. CRUD de usuarios (filtros por rol y estado, cambio de rol, desactivación) y de entidades responsables (con jurisdicción geoespacial opcional).

4 Entidad responsable

El usuario con rol `entity` pertenece a una entidad concreta y solo ve las incidencias asignadas a ella. Sus capacidades:

- Cola de incidencias asignadas, ordenadas por prioridad.
- Marcar como `in_progress`.
- Cerrar como `resolved` con nota obligatoria.
- Rechazar por falta de competencia.
- Comentarios oficiales visibles al autor y a los seguidores.

5 Accesibilidad

Toda la app es navegable por teclado y compatible con lectores de pantalla. Atajos:

- `Tab` / `Shift+Tab`: recorrer controles.
 - `Enter`: activar el control.
 - `Esc`: cerrar *modales*.
 - `+` / `-`: zoom in/out en el mapa.
-

Modelo de negocio preliminar

Análisis preliminar del modelo de negocio para una hipotética puesta en producción de EcoAlerta. No es parte del alcance técnico del TFG; lo incluyo para cumplir el OBJ-8.

Propuesta de valor

EcoAlerta aporta valor a tres actores distintos:

Ciudadanía. Canal ágil y transparente para reportar problemas medioambientales con seguimiento. Refuerzo del sentido de participación con el componente social.

Administraciones locales. Bajan el coste de recolección de incidencias y mejoran la calidad de los datos (foto, GPS, categoría). Panel de gestión integral, cuadro de mando para reportes a autoridades superiores.

Entidades especializadas (SEPRONA, bomberos, ONG). Cola priorizada de incidencias de su competencia, con evidencia que acelera la intervención.

Segmentos de mercado

Tres segmentos:

Ayuntamientos medianos (10 000–100 000 habitantes). Segmento principal. Concejalías de Medio Ambiente con poco desarrollo tecnológico propio. España tiene unos 2 500 municipios en este rango, mercado amplio.

Ayuntamientos grandes y diputaciones. Requieren integración con sistemas propios. Pocos clientes pero contratos unitarios mayores.

Espacios naturales protegidos y ONG. Parques naturales, reservas, confederaciones hidrográficas, asociaciones (SEO/BirdLife, Ecologistas en Acción, WWF España). Plataforma de recolección estandarizada para sus voluntarios.

Fuentes de ingresos

Modelo **B2G *freemium*** con tres planes. La aplicación es gratuita para el ciudadano (no hay modelo viable que cobre por reportar un problema medioambiental). Los ingresos vienen de las administraciones y organizaciones que contratan.

Tabla 1: Planes propuestos.

Plan	Precio	Incluye
Básico (gratis)	0 €/mes	Panel admin básico, hasta 100 incidencias/mes, 1 admin, mapa con marca de agua.
Municipal	150–300 €/mes	Incidencias ilimitadas, 5 admins, sin marca de agua, integración con email corporativo, exportación de informes, soporte por email.
Empresarial	600–1 200 €/mes	Multi-municipio, integración con Gestiona o GESCON, API personalizada, soporte prioritario, formación inicial, gamificación ciudadana personalizable.

Fuentes adicionales: consultoría de implantación inicial (tarifa puntual por puesta en marcha) y subvenciones europeas (Next Generation EU para digitalización municipal, programa LIFE de medio ambiente).

Estructura de costes

- **Infraestructura *cloud*:** 50–200 €/mes para un despliegue modesto (VPS, CDN, backups). Escala con el uso.
- **Almacenamiento de fotos:** ~0,02 €/GB/mes en S3 o equivalente. Crece linealmente con el volumen.
- **Dominio y certificados:** 20–100 €/año.
- **Personal:** en fases tempranas, un desarrollador a tiempo parcial. Con crecimiento, un equipo mínimo (1 *full-stack*, 1 soporte/ventas).
- **Cumplimiento legal:** registro de tratamientos RGPD, política de privacidad, posible DPO externo (~100 €/mes).

Viabilidad

Ejercicio sencillo a dos años: captación de 20 municipios en plan *Municipal* (200 €/mes medio) y 3 clientes *Empresarial* (900 €/mes medio) da unos 80 400 €/año a partir del mes 24, frente a costes de 50 000–60 000 €/año. Margen razonable para un proyecto autofinanciable tras una fase inicial de inversión.

El riesgo principal es el ciclo de venta largo del B2G (licitaciones, plazos administrativos). Mitigación: empezar con municipios pequeños mediante contratos menores (sin licitación) y casos de éxito replicables.

Aspectos legales

Cualquier puesta en producción debe abordar como mínimo:

- RGPD: base jurídica del tratamiento, política de privacidad, formulario de derechos, registro de actividades.
- Consentimiento explícito para geolocalización y foto.
- Condiciones de uso y política de moderación de comentarios.
- Acuerdos de Encargo de Tratamiento con las administraciones contratantes.
- Honor e imagen de personas retratadas en las fotos: procedimiento de borrado a petición y, si aplica, difuminado automático de rostros y matrículas.

Conclusión

El modelo es técnicamente viable y económicamente defendible, con ciclo de venta largo pero alta replicabilidad una vez probado en los primeros clientes. El producto desarrollado en este TFG da una base sólida para un piloto con un municipio dispuesto a colaborar, idealmente en la provincia de Alicante.
